Networks & Distributed Programming

Network programming: socket

Lectures 9-13



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



DIETI

UNINA

Application Layer

Creating Network Applications

- So far, we have looked at several network applications and protocols, we will now see **how to create network applications**.

- Most network applications include a **client-side** program and a **server-side** program that communicate through the network.

- When these two programs are executed, a **client process and a server process are created**, and these processes communicate with each other by reading from, and writing to, **sockets**.

Application Layer

Creating Network Applications

- Choice of protocols:
  - **Open protocols**. Protocols whose rules are public, so anyone can write either side and the two will interoperate.
  - **Proprietary protocols**. "Custom" protocols (not been openly published in an RFC or elsewhere). One team writes both sides and controls the format, and nobody else can write a program that talks to it.

- Examples:
  - A browser from one vendor talks to a server from another because both follow the same published rules.
  - An open protocol should use its well-known port; your own protocol must stay away from those.

Application Layer

Creating Network Applications

- Choice of protocols:
  - **Open protocols**. Protocols whose rules are public, so anyone can write either side and the two will interoperate.
  - **Proprietary protocols**. "Custom" protocols (not been openly published in an RFC or elsewhere).

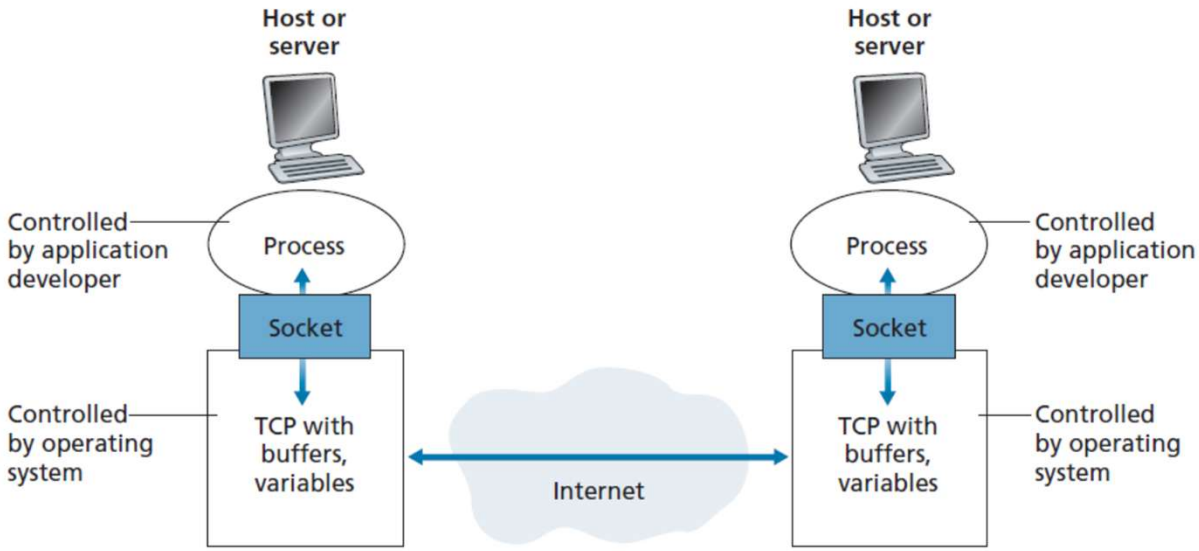
- There are two transport protocols that can be used:
  - **TCP** (Transmission Control Protocol), which is **connection oriented** and provides a reliable byte-stream channel through which data flows between two end systems.
  - **UDP** (User Datagram Protocol), which is **connectionless** and sends independent packets of data from one end system to the other, without any guarantees about delivery.

Socket

Creating Network Applications

- **Sockets** are a central element for the creation of network applications.

- The **Socket API** provides applications with access to the transport services implemented by the operating system's networking stack.



Socket

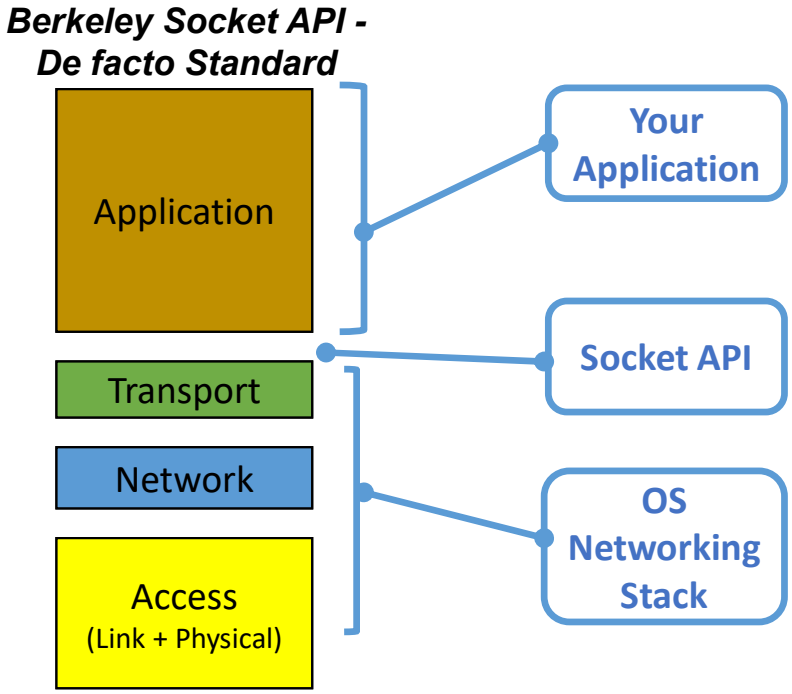
- A socket is one end of a two-way communication channel between two processes, referred to by a descriptor.

- Introduced in BSD 4.2 in 1983, and still the de facto standard: Unix, macOS, and Windows with a different spelling.
  - The two processes may be on the same machine or on two machines across the world: the calls do not change.
  - It is not tied to TCP/IP. The same API served other protocol families, and it is why the calls look generic.
  - Being a descriptor, it accepts read, write and close — the calls you already know.

Socket

Creating Network Applications

- **Sockets** are a central element for the creation of network applications.
- The **Socket API** provides applications with access to the transport services implemented by the operating system's networking stack.
- Several libraries, frameworks, and middleware systems wrap the Socket API, simplifying its use and providing higher-level communication abstractions.
- However, the basic socket API remains still widely used.

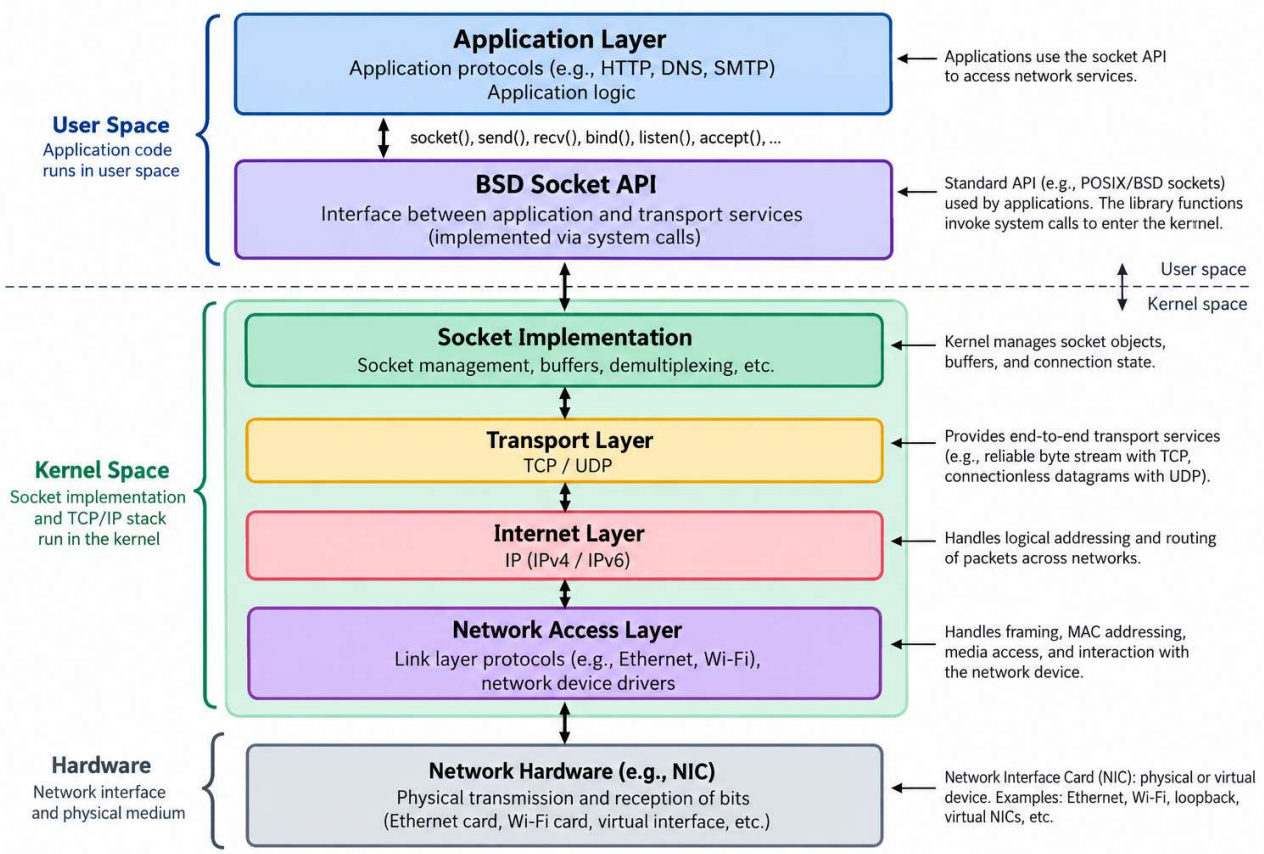


Where layers are implemented

Socket

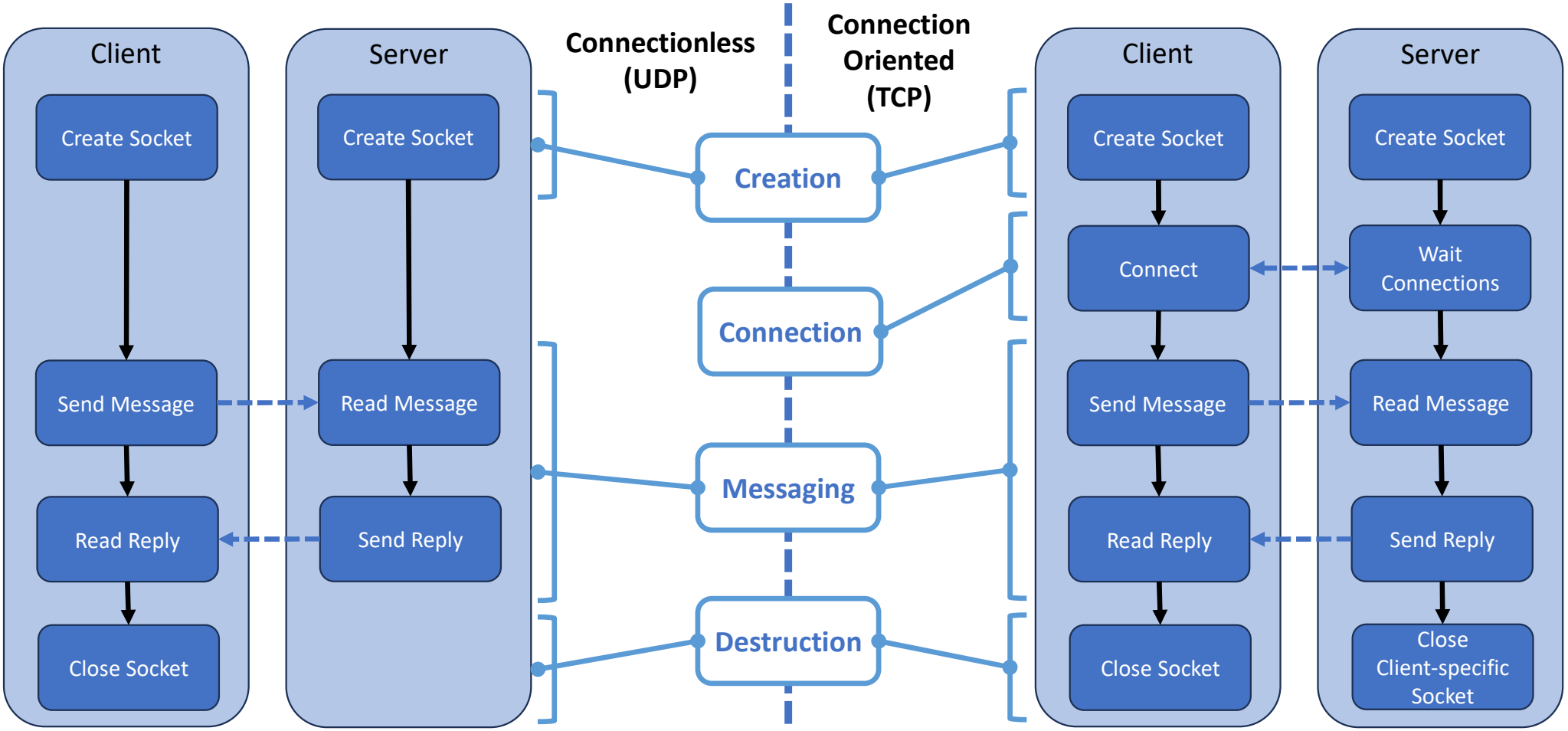
Creating Network Applications

- **Sockets** are a central element for the creation of network applications.



Socket

Connection-oriented vs. Connectionless



Application Layer
UDP and TCP

- UDP and TCP sockets implement the previous procedure to allow connectionless and connection-oriented communication, respectively.

- The basic APIs providing UDP and TCP sockets are typically available in almost all programming languages and operating systems:
  - C, C++, C#, Java, Perl, Python, Matlab, etc.

- The underlying implementation and the usage depends on the configuration of the machines.

- In Unix domain we use Berkeley sockets (aka BSD or POSIX sockets) for both TCP and UDP connections. These are native C APIs.

Socket

Sockets: Creation

- Sockets are created in C by using the socket() function:

```
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

Where:

- **domain:** specifies the network domain and addressing family (e.g., AF_INET for IPv4).
- **type:** specifies the communication semantics (e.g., stream or datagram).
  - SOCK_STREAM -> TCP socket
  - SOCK_DGRAM -> UDP socket
- **protocol:** specifies a particular protocol to be used.
  - This is often 0 for the default protocol.
- **sockfd:** is the file descriptor that identifies the newly created socket.
  - Notice that sockets adhere to the Unix philosophy that “everything is a file” where I/O sources are treated as files (and associated to a file descriptor).

Socket

Sockets: Creation

- Sockets are created in C by using the socket() function:

```
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

Domain — which family	
AF_INET	IPv4
AF_INET6	IPv6
AF_LOCAL	two processes on this machine, through the file system
Type — which kind of channel	
SOCK_STREAM	a byte stream: connected, ordered, reliable
SOCK_DGRAM	independent messages, no connection, no guarantees
SOCK_RAW	the packet built by hand, below the transport layer

Socket

Sockets: Closing

- Sockets can be closed by using the close() function:

```
#include <unistd.h>

int close(int socket-id);
```

Where:

- **socket-id:** is the file descriptor of the socket to close
- **val:** return value which is 0 on success, and -1 otherwise.

Socket

Sockets: Binding

- We can associate a socket to local address and port by using the bind() function:

```
#include <sys/socket.h>

int bind(int socket, const struct sockaddr *address, socklen_t address_len);
```

Where:

- **socket:** is the file descriptor of the socket we want to bind.
- **address:** pointer to a sockaddr structure specifying on which port and address to bind.
  - address.sin_addr often set to INADDR_ANY so connections from all addresses are listened (multiple IPs).
- **address_len:** specifies the length of the sockaddr structure pointed to by the address argument (we can use sizeof() to get it).
- val: return value which is 0 on success, and -1 otherwise.

- Notice that this function **is used on servers** as we must bind the socket to a specific port but is **optional on clients** (the OS assigns a free port automatically).

Sockets: Definitions

- Internet-domain sockets in C rely on some structure to define the addresses and ports of the sending/receiving hosts:

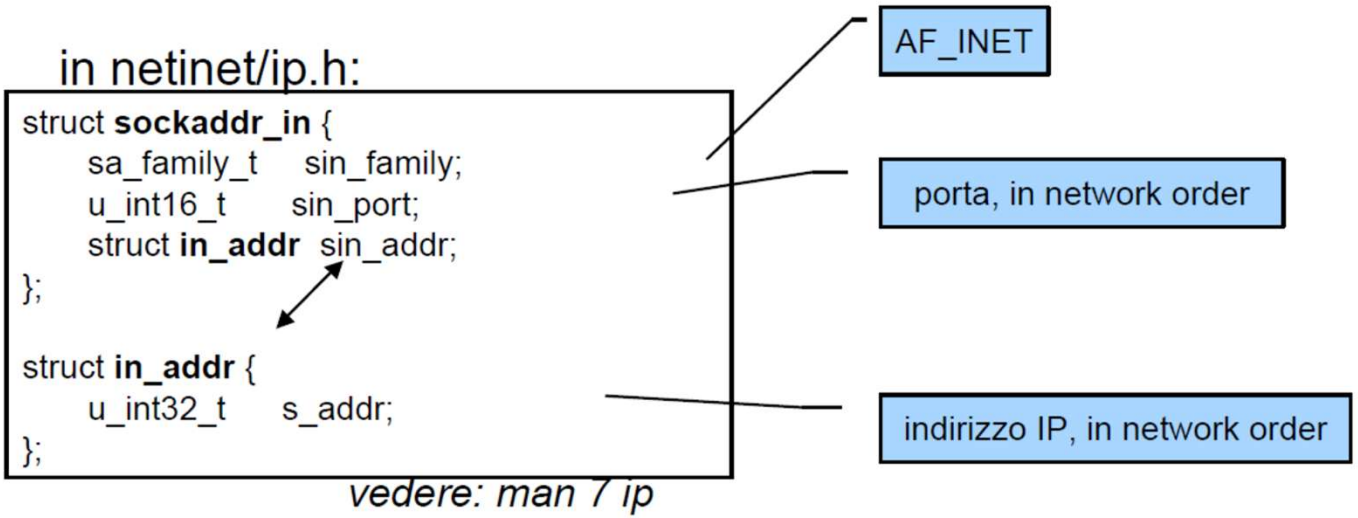
```
#include <netinet/in.h>
#include <arpa/inet.h>              // for inet_addr()
#include <sys/types.h>              // some custom types are defined here


struct sockaddr_in {
   short             sin_family;    // family of the address, typically set to AF_INET (IPv4)
   unsigned short    sin_port;      // port number, e.g. htons(3490)
   struct in_addr    sin_addr;      // see struct in_addr below
   char              sin_zero[8];   // typically zeros, now this structure has same size as sockaddr and can be casted to it
};

struct in_addr {
   unsigned long s_addr;  // IP address, can be loaded with inet_addr() or set to INADDR_ANY for localhost
};
```

Sockets: Definitions

- Internet-domain sockets in C rely on some structure to define the addresses and ports of the sending/receiving hosts:



Sockets: Definitions

- Internet-domain sockets in C rely on some structure to define the addresses and ports of the sending/receiving hosts:
  - Identificati da:
    1) un indirizzo IP (intero a 32 bit, es. 143.225.5.3)
    2) una porta (intero a 16 bit, da 0 a 65535)
  - Porte:
    - per offrire diversi servizi dallo stesso indirizzo IP
    - da 0 a 1023: porte riservate (ai processi di root)
    - da 5000 a 32768: porte utente
    - altre: porte effimere (per i client, ai quali non interessa scegliere una porta specifica)

Sockets: ip address and port

- Internet-domain sockets in C rely on some structure to define the addresses and ports of the sending/receiving hosts:

  - Come viene memorizzato un intero in memoria?
    - sono usati 4 bytes
    - consideriamo 65537, cioe' $2^{16} + 1$, all'indirizzo 100
    - codifica “Big Endian” (prima il byte *piu'* significativo):

      00000000 00000000 10000000 00000001
      100 101 102 103

    - codifica “Little Endian” (prima il byte *meno* significativo):

      00000001 10000000 00000000 00000000
      100 101 102 103

Sockets: ip address and port

- Internet-domain sockets in C rely on some structure to define the addresses and ports of the sending/receiving hosts:

  - Come viene memorizzato un intero in memoria?
    - sono usati 4 bytes
    - consideriamo 65537, cioe' $2^{16} + 1$, all'indirizzo 100
    - codifica “Big Endian” (prima il byte *piu'* significativo):

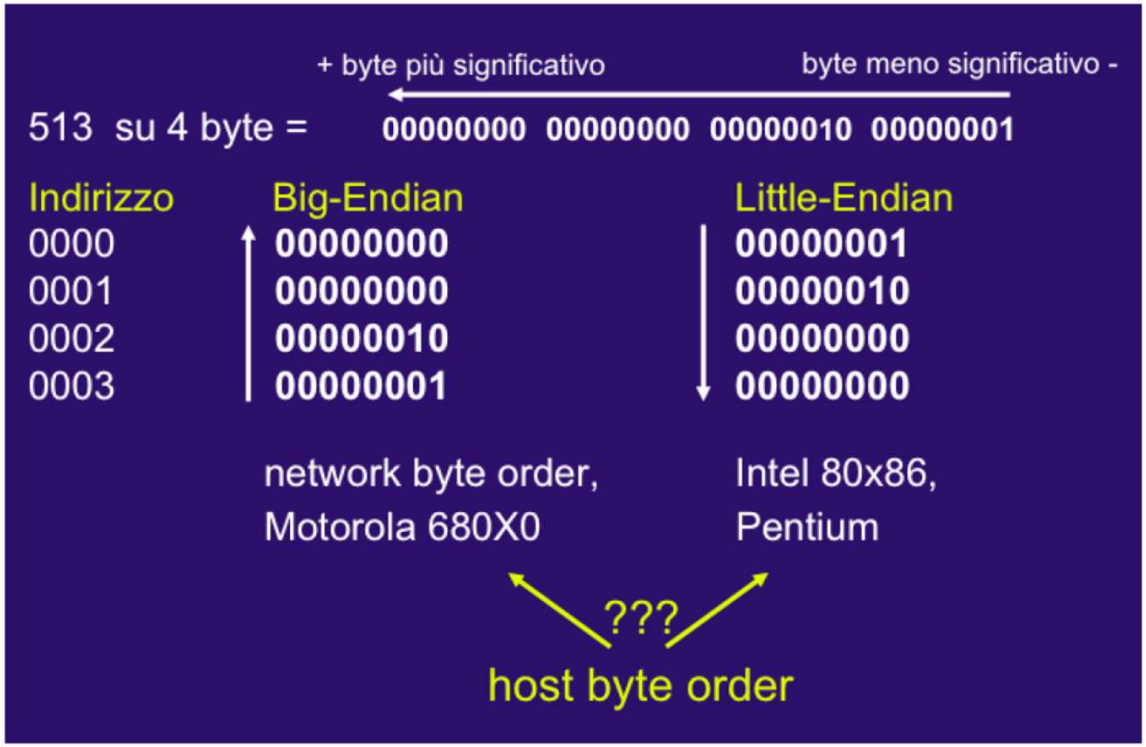
      00000000 00000000 10000000 00000001
      100 101 102 103

    - codifica “Little Endian” (prima il byte *meno* significativo):

      00000001 10000000 00000000 00000000
      100 101 102 103

Sockets: ip address and port

- Internet-domain sockets in C rely on some structure to define the addresses and ports of the sending/receiving hosts:



Sockets: ip address and port

- Internet-domain sockets in C rely on some structure to define the addresses and ports of the sending/receiving hosts:
  - i processori possono essere di entrambi i tipi
    - i protocolli TCP/IP lavorano con **big endian**
    - sono necessarie funzioni di **conversione**
  - conversioni tra unsigned:
    - #include <netinet/in.h>
    - uint32_t **h**to**n**l(uint32_t x)
    - uint16_t **h**to**n**s(uint16_t x)
    - uint32_t **n**to**h**l(uint32_t x)
    - uint16_t **n**to**h**s(uint16_t x)

h	= Host
n	= Network (big endian)
l	= Long (4 bytes)
s	= Short (2 bytes)

Sockets: ip address and port

- Internet-domain sockets in C rely on some structure to define the addresses and ports of the sending/receiving hosts:
  - **Dotted notation**

```
struct sockaddr_in indirizzo;

indirizzo.sin_family = AF_INET;

if (inet_pton(AF_INET, "143.225.5.3", &indirizzo.sin_addr) != 1)
{
   perror("inet_pton");
   exit(1);
}
```

  - inet_pton converts IPv4 / IPv6 in binary format
    - It fills in_addr (IPv4) or in6_addr (IPv6)
    - outcome:
      - 1 = OK
      - 0 = not valid strings
      - -1 = error (es. not supported family)

Sockets: ip address and port

- Example ipv4 address

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h> //header POSIX che fa parte dello stack di rete BSD-style

#define MYPORT 4242

int main(void) {
    struct sockaddr_in my_addr;
    memset(&my_addr, 0, sizeof(my_addr)); // azzera tutta la struttura

    my_addr.sin_family = AF_INET; // host byte order

    my_addr.sin_port = htons(MYPORT); // short → network byte order

if (inet_pton(AF_INET, "10.12.110.57", &my_addr.sin_addr) != 1) {
        perror("inet_pton"); exit(1);
    }
    return 0;
}
```

Sockets: ip address and port

- Example ipv6 address

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>

int main(void) {
    struct sockaddr_in6 indirizzo; //indirizzo a IPv6 (a 16 byte)

    memset(&indirizzo, 0, sizeof(indirizzo)); // azzera la struttura
    indirizzo.sin6_family = AF_INET6;

    if (inet_pton(AF_INET6, "2001:db8::1234", &indirizzo.sin6_addr) != 1) {
        perror("inet_pton"); exit(1);
    }

    indirizzo.sin6_port = htons(8080); // porta

    return 0;
}
```

Application Layer

Sockets: Definitions

- Ipv4 vs ipv6 addresses

```
struct sockaddr_in {
    sa_family_t     sin_family;
    in_port_t       sin_port;
    struct in_addr  sin_addr;   // 4 B
    unsigned char   sin_zero[8];
};                          // 16 B totali

struct sockaddr_in6 {
    sa_family_t     sin6_family;
    in_port_t       sin6_port;
    uint32_t        sin6_flowinfo;
    struct in6_addr sin6_addr;  // 16 B
    uint32_t        sin6_scope_id;
};                          // ~28–32 B totali
```

Application Layer

Sockets: Definitions

- Ipv4 address binding

```
#include <netinet/in.h>
#include <arpa/inet.h>

struct sockaddr_in mio_indirizzo;

// azzera tutto
memset(&mio_indirizzo, 0, sizeof(mio_indirizzo));

mio_indirizzo.sin_family = AF_INET;
mio_indirizzo.sin_port   = htons(5200);      // network byte order
mio_indirizzo.sin_addr.s_addr = htonl(INADDR_ANY);

if (bind(fd, (struct sockaddr *)&mio_indirizzo, sizeof(mio_indirizzo)) < 0) {
    perror("bind");
    exit(1);
}
```

Application Layer
Sockets: Definitions

- Ipv6 address binding

```
#include <netinet/in.h>
#include <arpa/inet.h>

struct sockaddr_in mio_indirizzo;

struct sockaddr_in6 mio_ind6 = {0};

mio_ind6.sin6_family = AF_INET6;
mio_ind6.sin6_port   = htons(5200);
mio_ind6.sin6_addr   = in6addr_any;   // costante globale IPv6

bind(fd, (struct sockaddr *)&mio_ind6, sizeof(mio_ind6));
```

Sockets: UDP

UDP (**User Datagram Protocol**) is a simple transport-layer protocol. Unlike TCP, which is connection-oriented, UDP does not establish a connection between client and server and does not provide reliability guarantees.

While TCP transmits a stream of bytes, UDP sends discrete packets called **datagrams**.

Because UDP does not require connection establishment or termination handshakes, it introduces less overhead than TCP and can be faster for transmitting small amounts of data.

Transmission errors and packet loss are relatively infrequent in many networks, so when occasional data loss is acceptable, UDP can provide very fast and efficient communication.

Sockets: Send and Receive (UDP)

- Processes of sending or receiving messages in UDP are performed by the sendto() and recvfrom() functions:

```
#include <sys/socket.h>

int ob = sendto(int osock, const void *obuf, size_t olen, int oflags, const struct sockaddr *oaddr, socklen_t oaddr_len);
```

```
#include <sys/socket.h>

int ib = recvfrom(int isock, void *ibuf, size_t ilen, int iflags, struct sockaddr *iaddr, socklen_t *iaddr_len);
```

Where:

- **osock/isock:** are the file descriptors on which to send/receive a message.
- **obuf/ibuf:** are pointers to the buffers containing a message to send/receive.
- **olen/ilen:** are lengths of the messages in bytes.
- **oflags/iflags:** specifies flags (typical values are 0 or MSG_WAITALL to wait until all olen/ilen bytes are sent/received).
- **oaddr/iaddr:** pointers to sockaddr structures containing the receiving/sending address.
- **oaddr_len/iaddr_len:** length in bytes of the sockaddr structure.
- **ob/ib:** number of bytes that are actually sent/received.

Sockets: Send and Receive (UDP)

```
ssize_t sendto(int sockfd, const void *buf, size_t len,
               int flags, const struct sockaddr *dest_addr,
               socklen_t addrlen);
```

- Return value:
  - Number of bytes sent, or -1 on error.

Blocking: By default, sendto() may block until the datagram is accepted by the OS. It does not wait for delivery to the receiver. MSG_DONTWAIT prevents blocking.

- Parameters:
  - **sockfd:** socket descriptor.
  - **buf:** pointer to the data buffer to be sent.
  - **len:** number of bytes to send.
  - **flags:** controls the sending behavior:
    - 0: default behavior.
    - MSG_DONTWAIT: performs a non-blocking send operation.
  - **dest_addr:** pointer to the destination address structure.
  - **addrlen:** size of the destination address structure (in bytes).

Sockets: Send and Receive (UDP)

```
ssize_t recvfrom(int sockfd, void *buf, size_t len,
                 int flags, struct sockaddr *src_addr,
                 socklen_t *addrlen);
```

Blocking: By default, recvfrom() blocks until a datagram is available. To avoid waiting indefinitely, use MSG_DONTWAIT or configure a receive timeout (SO_RCVTIMEO).

- Number of bytes received, or -1 on error.
- Parameters:
  - **sockfd:** socket descriptor.
  - **buf:** pointer to the receive buffer.
  - **len:** maximum number of bytes to receive. If the datagram is larger than the buffer, excess bytes are normally discarded.
  - **flags:** controls the receiving behavior:
    - 0: default behavior.
    - MSG_PEEK: reads a datagram without removing it from the receive queue.
    - MSG_DONTWAIT: performs a non-blocking receive operation.
  - **src_addr:** pointer to a structure that stores the sender's address.
  - **addrlen:** pointer to the address structure length (in bytes).

Sockets: Send and Receive (UDP)

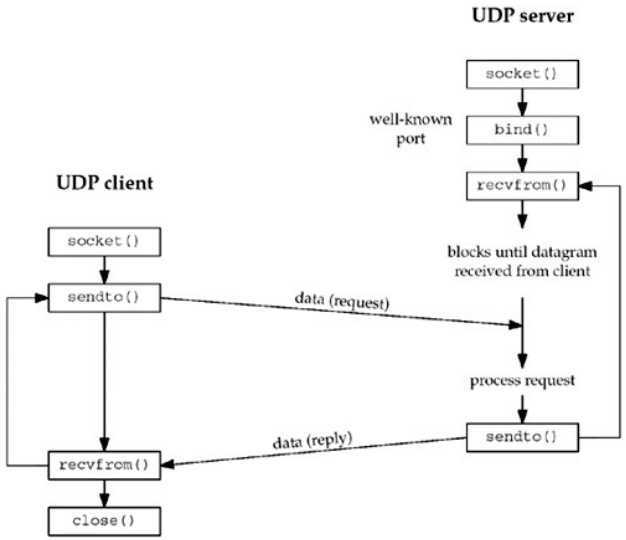
Both sendto() and recvfrom() accept a **flags** parameter that controls how data is sent or received.

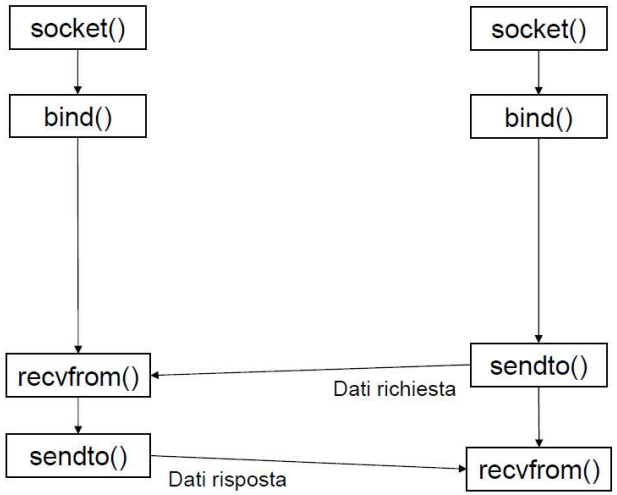
Flag	Description
0	Default behavior: no special options.
MSG_DONTWAIT	Performs a non-blocking operation; returns immediately if the operation would otherwise block.
MSG_PEEK	Receives a datagram without removing it from the receive queue.
MSG_TRUNC	On Linux, allows retrieving the full datagram length even if the receive buffer is too small.
MSG_WAITALL	Requests waiting for all specified bytes. Mainly useful with TCP; it does not combine multiple UDP datagrams.

Sockets: UDP

UDP (**User Datagram Protocol**) is a simple transport-layer protocol. Unlike TCP, which is connection-oriented, UDP does not establish a connection between client and server and does not provide reliability guarantees.

```
(sockfd = socket(AF_INET, SOCK_DGRAM, 0)
```





Socket Programming (Simple Example C)

```c
// client-side (includes omitted)

int main() {
   int sockfd;
   char buffer[1024];
   const char *hello = "Hello from client";
   struct sockaddr_in servaddr;

   if ( (sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0 ) {
      perror("socket creation failed");
      exit(EXIT_FAILURE);
   }

   memset(&servaddr, 0, sizeof(servaddr));

   servaddr.sin_family = AF_INET;
   servaddr.sin_port = htons(8080);
    inet_pton(AF_INET, "192.168.1.10", &servaddr.sin_addr);

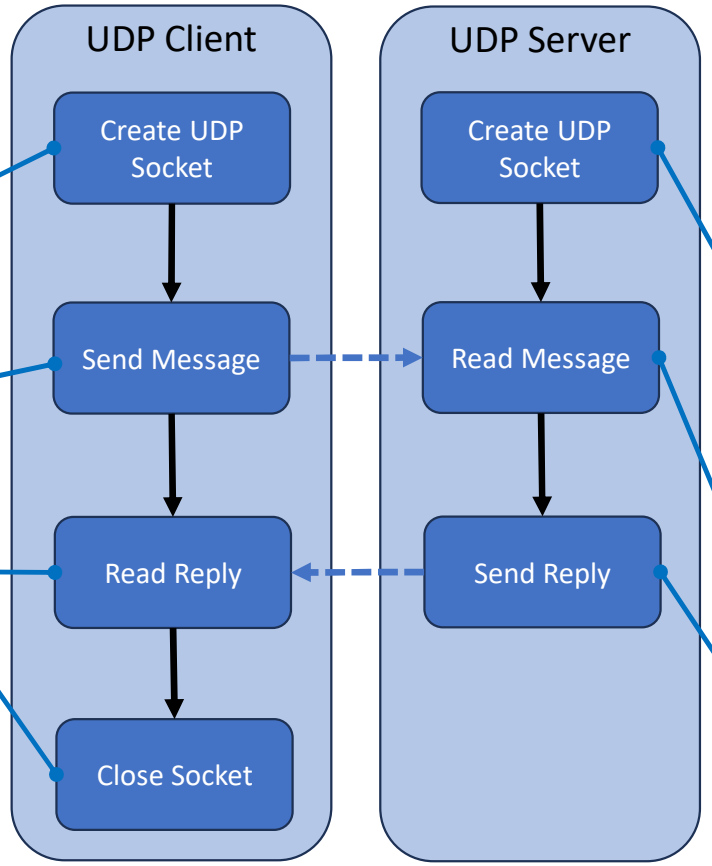
   int n;
   socklen_t len;

   sendto(sockfd, (const char *)hello, strlen(hello),
      0, (const struct sockaddr *) &servaddr, sizeof(servaddr));
   printf("Hello message sent.\n");

   n = recvfrom(sockfd, (char *)buffer, 1024,
            0, (struct sockaddr *) &servaddr, &len);
   buffer[n] = '\0';
   printf("Received \"%s\"\n", buffer);

   close(sockfd);

   return 0;
}
```




```c
//server-side (includes omitted)

int main() {
   int sockfd;
   char buffer[1024];
   const char *hello = "Hello from server";
   struct sockaddr_in servaddr, cliaddr;

   if ( (sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0 ) {
      perror("socket creation failed");
      exit(EXIT_FAILURE);
   }

   memset(&servaddr, 0, sizeof(servaddr));
   memset(&cliaddr, 0, sizeof(cliaddr));

   servaddr.sin_family    = AF_INET;
   servaddr.sin_addr.s_addr = INADDR_ANY;
   servaddr.sin_port = htons(8080);

   if ( bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0 ) {
      perror("bind failed");
      exit(EXIT_FAILURE);
   }

   socklen_t len = sizeof(cliaddr);
   int n;

   n = recvfrom(sockfd, (char *)buffer, 1024,
            0, ( struct sockaddr *) &cliaddr, &len);
   buffer[n] = '\0';
   printf("Received \"%s\"\n", buffer);

   sendto(sockfd, (const char *)hello, strlen(hello),
      0, (const struct sockaddr *) &cliaddr, len);
   printf("Hello message sent.\n");

   close(sockfd);
   return 0;
}
```

Sockets: Send and Receive (UDP)

- Minimal UDP Server

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    int s;
    struct sockaddr_in addr = {0};
    char buf[1024];

    s = socket(AF_INET, SOCK_DGRAM, 0);

    addr.sin_family      = AF_INET;
    addr.sin_port        = htons(5200);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    bind(s, (struct sockaddr *)&addr, sizeof(addr));
```

Sockets: Send and Receive (UDP)

- Minimal UDP Server

```
while (1) {
        struct sockaddr_in from = {0};
        socklen_t flen = sizeof(from);

        int n = recvfrom(s, buf, sizeof(buf), 0,
                        (struct sockaddr *)&from, &flen);

        sendto(s, buf, n, 0,
              (struct sockaddr *)&from, flen);
    }

    close(s);
}
```

Sockets: Send and Receive (UDP)

- Minimal UDP Client

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int s;
    struct sockaddr_in addr = {0};
    char buf[1024] = "ciao";

    s = socket(AF_INET, SOCK_DGRAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port   = htons(5200);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
```

Sockets: Send and Receive (UDP)

- Minimal UDP Client

```
sendto(s, buf, strlen(buf), 0,
          (struct sockaddr *)&addr, sizeof(addr));

    int n = recvfrom(s, buf, sizeof(buf), 0, NULL, NULL);

    /* ... usa buf ... */
    buf[n] = '\0';
   printf("ricevuto dal server: %s\n", buf);

    close(s);
}
```

Sockets: Implicit Binding (UDP)

When a UDP socket is created with socket(), it is not yet bound to a local port.

- If the client calls sendto() without first calling bind(), the operating system automatically assigns an available local port (implicit binding).
- The source IP address is selected according to the network interface and routing configuration.
- The assigned port remains associated with the socket until it is closed.
- A subsequent call to bind() on the same socket fails because the socket is already bound.

Note: implicit binding does not establish a connection between the client and the server.

Sockets: Implicit Binding (UDP)

When a UDP socket is created with socket(), it is not yet bound to a local port.

- If the client calls sendto() without first calling bind(), the operating system automatically assigns an available local port (implicit binding).
- The source IP address is selected according to the network interface and routing configuration.
- The assigned port remains associated with the socket until it is closed.
- A subsequent call to bind() on the same socket fails because the socket is already bound.

```
sockfd = socket(AF_INET, SOCK_DGRAM, 0);

// Implicit binding
sendto(sockfd, message, strlen(message), 0,
       (struct sockaddr *)&servaddr, sizeof(servaddr));

// Attempt to bind an already bound socket
bind(sockfd, (struct sockaddr *)&cliaddr,
     sizeof(cliaddr));  // ERROR!
```

UDP: Server and Client Start-up Order

Case 1: Client starts before Server

- The client sends a datagram using sendto().
- If a UDP datagram reaches the destination host but no socket is bound to the destination port, the UDP layer discards the datagram. An ICMP Port Unreachable message may be generated.
- When the server starts, it cannot retrieve the lost datagram.
- The client may remain blocked on recvfrom() or receive an error (e.g., ECONNREFUSED).

Case 2: Server starts before Client

- The server calls bind() and waits on recvfrom().
- The client starts and sends a datagram.
- The server receives the datagram and sends a reply.
- The client receives the reply.

Note: UDP does not buffer datagrams for applications that are not listening. Lost datagrams are not automatically retransmitted.

UDP: Receive Timeout

By default, recvfrom() is a blocking operation: it waits until a datagram is received or an error occurs.

A receive timeout can be configured using setsockopt() with the SO_RCVTIMEO option.

Example: timeout

```
struct timeval timeout;
timeout.tv_sec = 5;
timeout.tv_usec = 0;

setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO,
        &timeout, sizeof(timeout));
```

Behavior:

- If a datagram arrives before the timeout, recvfrom() returns the received data.
- If no datagram arrives within 5 seconds, recvfrom() returns -1, with errno set to EAGAIN or EWOULDBLOCK.
- The timeout does not retransmit lost datagrams.

UDP: Communication with Multiple Clients

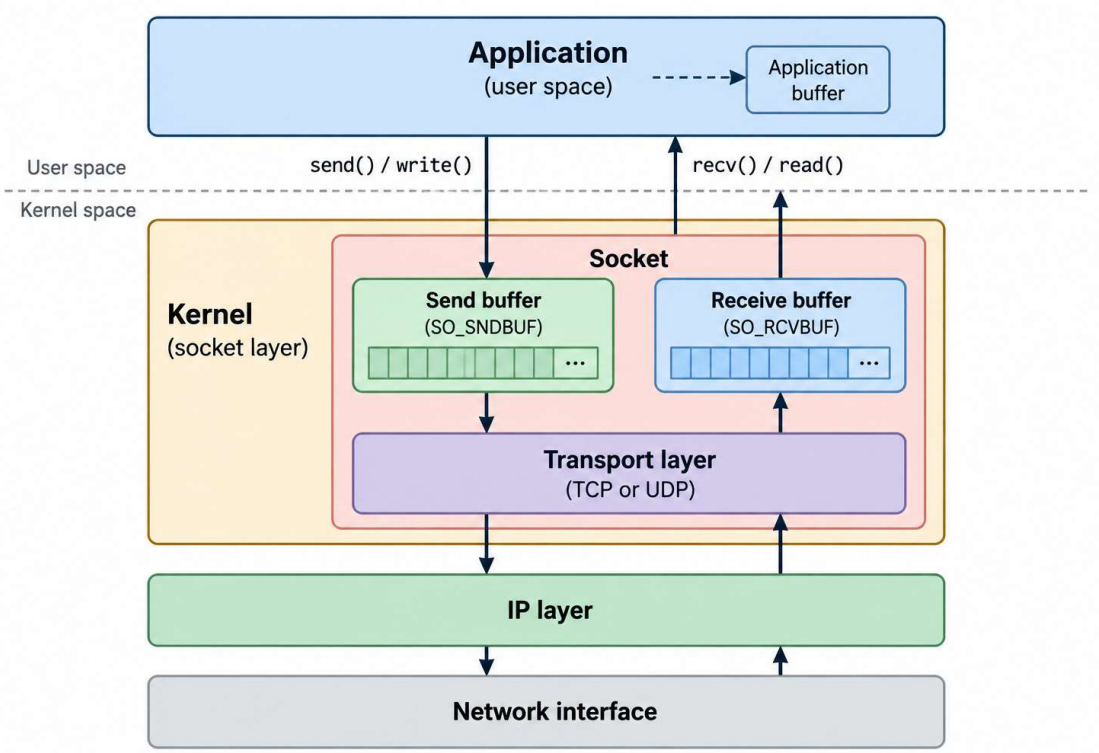
UDP does not require a connection to be established between the client and the server.

- A UDP server uses a single socket bound to a known port.
- Multiple clients can send datagrams to the same server socket.
- Each recvfrom() call receives a datagram and provides the sender's IP address and port number.
- The server uses sendto() to send a reply to the corresponding client.
- No listen(), accept(), or separate socket for each client is required.

Note: A UDP server can communicate with multiple clients using a single socket, without establishing connections.

Sockets UDP: Buffer

UDP sockets have two kernel-managed buffers: a send buffer for outgoing datagrams and a receive buffer for incoming datagrams.



Sockets UDP: Buffer

UDP sockets have two kernel-managed buffers: a send buffer for outgoing datagrams and a receive buffer for incoming datagrams.

Sending — sendto()

- The send buffer size can be configured using SO_SNDBUF.
- Each sendto() call sends a complete UDP datagram.
  - **Blocking socket (default):** if insufficient buffer space is available, sendto() may block until the datagram can be accepted.
  - **Non-blocking socket:** sendto() returns immediately; if the datagram cannot be accepted, it returns -1 with EAGAIN or EWOULDBLOCK.

UDP does not support partial datagram sends: the complete datagram is accepted or the operation fails.

Sockets UDP: Buffer

UDP sockets have two kernel-managed buffers: a send buffer for outgoing datagrams and a receive buffer for incoming datagrams.

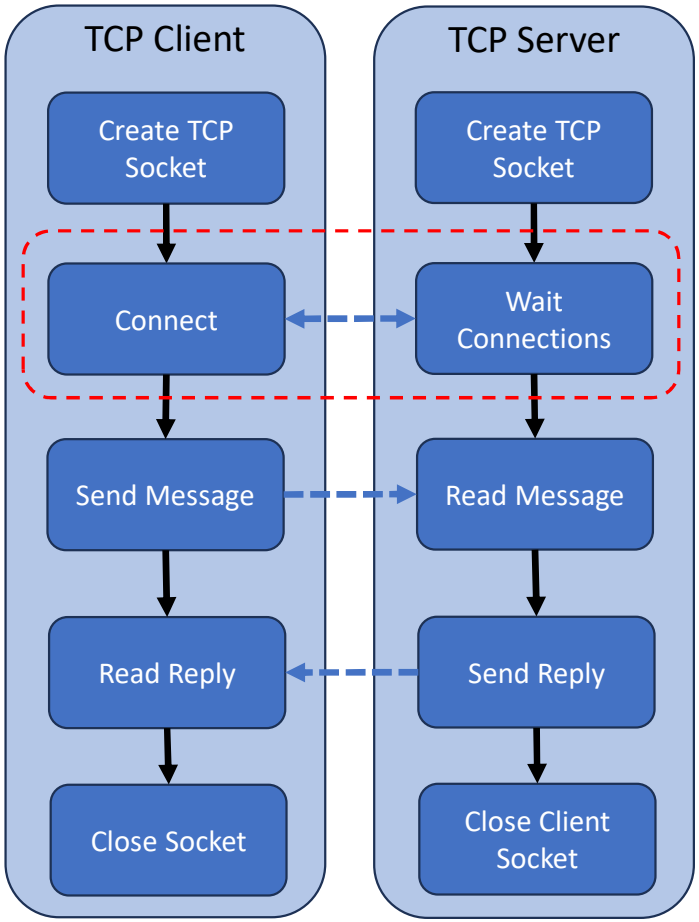
Receiving — recvfrom()

- The receive buffer size can be configured using SO_RCVBUF.
- Incoming datagrams are queued in the socket's receive buffer.
  - **Blocking socket (default):** recvfrom() blocks if no datagram is available.
  - **Non-blocking socket:** recvfrom() returns immediately with EAGAIN or EWOULDBLOCK if no datagram is available.
- Each recvfrom() retrieves at most one datagram.

If the application buffer is smaller than the received UDP datagram, the datagram is truncated: the excess bytes are discarded and cannot be retrieved by subsequent receive calls.

Socket: From UDP to TCP

- Unlike UDP, in TCP client and server need to **handshake** before messages can be transmitted.
- We use **two sockets on the server-side**:
  - A **welcoming socket**, which is always on, and allows clients to perform handshakes with the server.
  - A **client-specific socket**, which is created after the handshake, and is used by client and server to communicate.
- The three-way handshake, which takes place within the transport layer, is completely **invisible to the client and the server programs**.
- Once the connection is accepted by the server (handshake success) **the communication moves to the newly created socket**.



Sockets: Connection (TCP)

- Client-side TCP connection is performed by using the connect() function:

```
#include <sys/socket.h>

int connect(int socket, const struct sockaddr *address, socklen_t address_len);
```

Where:

- **socket:** is the file descriptor of the socket we use to connect.
- **address:** pointer to a sockaddr structure containing the server address.
- **address_len:** specifies the length of the sockaddr structure pointed to by the address argument (we can use sizeof() to get it).
- **val:** return value which is 0 on success, and -1 otherwise.

Note: connect() can also be used with UDP. Unlike TCP, it does not establish a connection, it only sets a default destination address and filters incoming datagrams.

Sockets: Connection (TCP only)

- Client-side TCP connection is performed by using the connect() function:
  - ipv4

```
#include <netinet/in.h>   // sockaddr_in, htons
#include <arpa/inet.h>    // inet_pton
#include <string.h>       // memset
#include <stdio.h>        // perror
#include <stdlib.h>       // exit

/* ...creazione socket => fd ... */

struct sockaddr_in indirizzo;
memset(&indirizzo, 0, sizeof(indirizzo)); // azzera la struttura

indirizzo.sin_family = AF_INET;
indirizzo.sin_port = htons(5200);

if (inet_pton(AF_INET, "143.225.5.3", &indirizzo.sin_addr) != 1) {
    perror("inet_pton"); exit(1);}

if (connect(fd, (struct sockaddr *)&indirizzo, sizeof(indirizzo)) < 0) {
    perror("connect"); exit(1);
}
```

Sockets: Connection (TCP only)

- Client-side TCP connection is performed by using the connect() function:
  - ipv6

```
struct sockaddr_in6 indirizzo6 = {0};

indirizzo6.sin6_family = AF_INET6;
indirizzo6.sin6_port   = htons(5200);

if (inet_pton(AF_INET6, "2001:db8::1234", &indirizzo6.sin6_addr) != 1) {
    perror("inet_pton");
    exit(1);
}

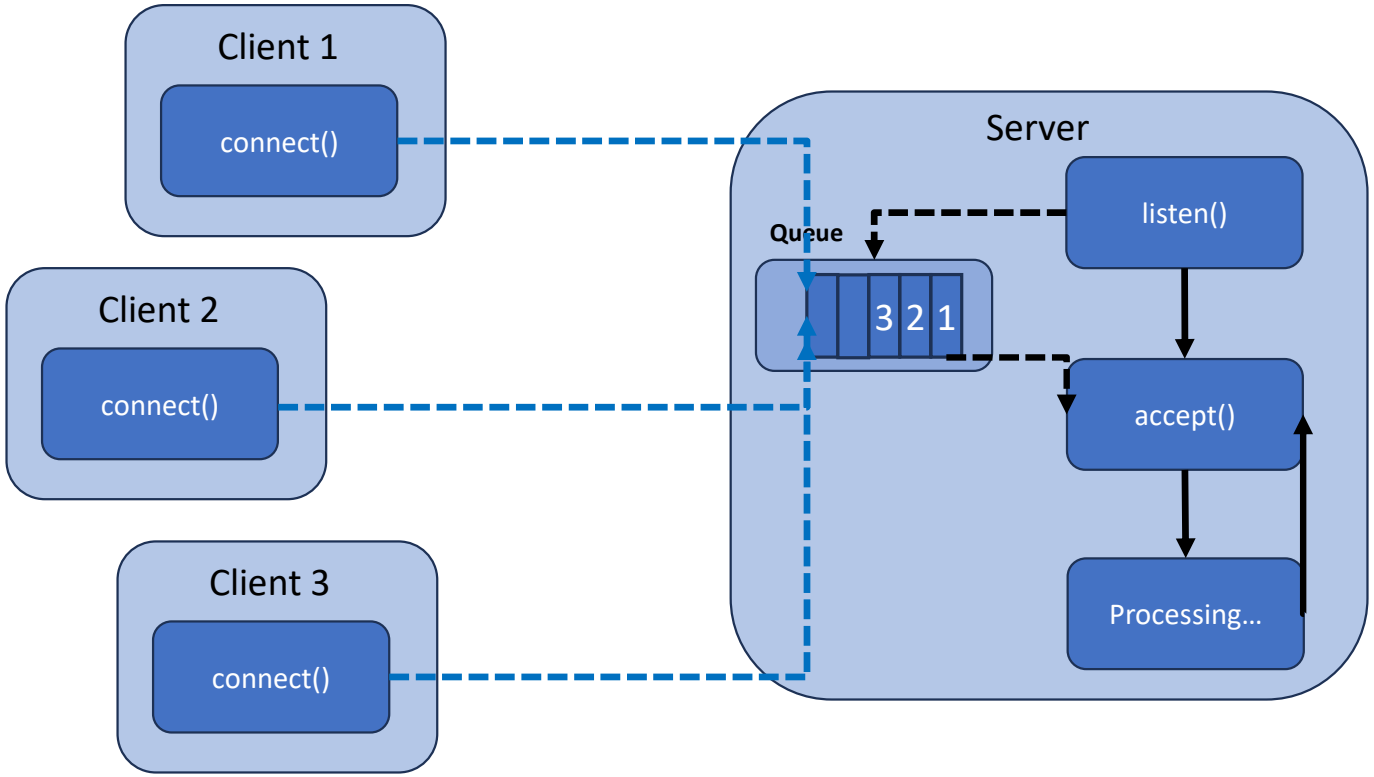
connect(fd, (struct sockaddr *)&indirizzo6, sizeof(indirizzo6));
```

Sockets: Wait-for-connection (TCP only)

- Server-side TCP wait-for-connection is performed through two functions: listen() and accept(), which works in combination with the clients connect().

- The listen() opens a queue where incoming connections are stored.

- The accept() opens the client-specific socket.



Sockets: Wait-for-connection (TCP only)

- The listening socket is created once, bound to the known port, and never carries data.
- Every accept returns a connection socket, dedicated to one client and useless to any other.
- Data flows only on the second kind. The first one keeps listening while the conversation happens elsewhere.
- E.g., a server serving three clients holds four descriptors

The three-way handshake that creates the connection happens in the transport layer, and is invisible to both programs.

Sockets: Wait-for-connection (TCP only)

- Server-side TCP wait-for-connection is performed by the listen() and accept() functions:

```
#include <sys/socket.h>

int listen(int socket, int backlog);
int accept(int socket, struct sockaddr *address, socklen_t *address_len);
```

Where:

- **socket:** is the file descriptor of the socket on which to wait for connections (must be bound to address/port).
- **backlog:** maximum length of the queue of pending connections, how many completed connections may wait before the kernel starts refusing or ignoring them.
- **address:** can be a null pointer, or a pointer to a sockaddr structure where the address of the connecting socket shall be returned.
- **address_len:** length of the address.
- **new_sockfd:** new socket descriptor on which the client and the server will communicate.
- **val:** return value which is 0 on success, and -1 otherwise.

Sockets: Wait-for-connection (TCP only)

- Server-side TCP wait-for-connection is performed by the listen() and accept() functions:

int listen(int sockfd, int backlog);

- Marks a socket as a **listening socket**, ready to accept incoming connections.
- Used with connection-oriented socket types, such as SOCK_STREAM and SOCK_SEQPACKET (protocol SCTP message-oriented).
- **backlog:** specifies the maximum number of pending connections waiting to be accepted.
  - If the queue is full, new connection attempts may be delayed or rejected.
- **Return value:** 0 on success, -1 on error.

Sockets: Wait-for-connection (TCP only)

- Server-side TCP wait-for-connection is performed by the listen() and accept() functions:

```
int accept(int sockfd,
        struct sockaddr *indirizzo_client,
        socklen_t *dimensione_indirizzo);
```

- Il secondo e terzo argomento servono ad identificare il client
  - possono essere NULL
- Restituisce un nuovo descrittore! (oppure -1)
  - crea un nuovo socket, dedicato a questa nuova connessione
  - il vecchio socket resta in ascolto
- Blocca il processo se non vi sono connessioni in attesa
  - Il socket puo' essere marcato non-blocking.

Sockets: TCP

- client code

```c
int main(void) {
    char buf[256];
    struct sockaddr_in srv;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
    memset(&srv, 0, sizeof srv);
    srv.sin_family = AF_INET;
    srv.sin_port   = htons(5000);
    inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);   /* not INADDR_ANY */
    if (connect(fd, (struct sockaddr *) &srv, sizeof srv) < 0) {
        perror("connect"); exit(EXIT_FAILURE); }
    write(fd, "hello\n", 6);
    ssize_t n = read(fd, buf, sizeof buf - 1);  /* room for the '\0' */
    if (n > 0) { buf[n] = '\0'; printf("%s", buf); }
    close(fd);
}
```

Sockets: TCP

- server code

```
1  struct sockaddr_in me;
2  int yes = 1;
3  int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
4  if (listen_fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
5  setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
6  memset(&me, 0, sizeof me);
7  me.sin_family      = AF_INET;
8  me.sin_port        = htons(5000);
9  me.sin_addr.s_addr = htonl(INADDR_ANY);   /* any local interface */
10 if (bind(listen_fd, (struct sockaddr *) &me, sizeof me) < 0) {
11     perror("bind"); exit(EXIT_FAILURE); }
12 if (listen(listen_fd, 16) < 0) {
13     perror("listen"); exit(EXIT_FAILURE); }
```

SO_REUSEADDR is what lets you restart the server immediately instead of waiting for the old port to be released.

Sockets: Options

Socket options can be read and configured using getsockopt() and setsockopt().

```
#include <sys/socket.h>

int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);

int setsockopt(int sockfd, int level, int optname,  const void *optval, socklen_t optlen);
```

Parameters:
sockfd: socket descriptor.
level: protocol level at which the option is defined:
- SOL_SOCKET: socket-level options
- IPPROTO_TCP: TCP options
- IPPROTO_IP: IPv4 options

optname: identifies the socket option.
optval: pointer to the option value.
optlen: size of the option value in bytes. In getsockopt(), it is a pointer to an input/output length.
Return value: 0 on success, -1 on error (errno is set).

Sockets: Options SO_REUSEADDR

- **SO_REUSEADDR** is a socket-level option (SOL_SOCKET).
- It allows a local address and port to be reused under certain conditions, particularly when previous TCP connections are in **TIME_WAIT**.

- The option value (optval) is a pointer to an integer:
  - Non-zero value: enables the option.
  - Zero value: disables the option.
  - optlen is set to sizeof(int).

Example:

```
int opt = 1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

The option must be set before calling bind().

Sockets: Options SO_REUSEADDR

Context:

- After a TCP connection is closed, the endpoint performing the active close typically enters the TIME_WAIT state for a limited period.
- TIME_WAIT helps prevent delayed packets from an old connection from interfering with a new connection.
- Connections in TIME_WAIT may prevent a new socket from binding to the same local address and port.
- SO_REUSEADDR allows the address and port to be reused, subject to operating system rules.

When is it useful?

- When restarting a TCP server shortly after it has been stopped, bind() may fail with Address already in use.
- Enabling SO_REUSEADDR can allow the server to restart without waiting for old connections to expire.

Note: SO_REUSEADDR does not normally allow two TCP servers to listen on the same IP address and port simultaneously.

Sockets: Options SO_REUSEADDR

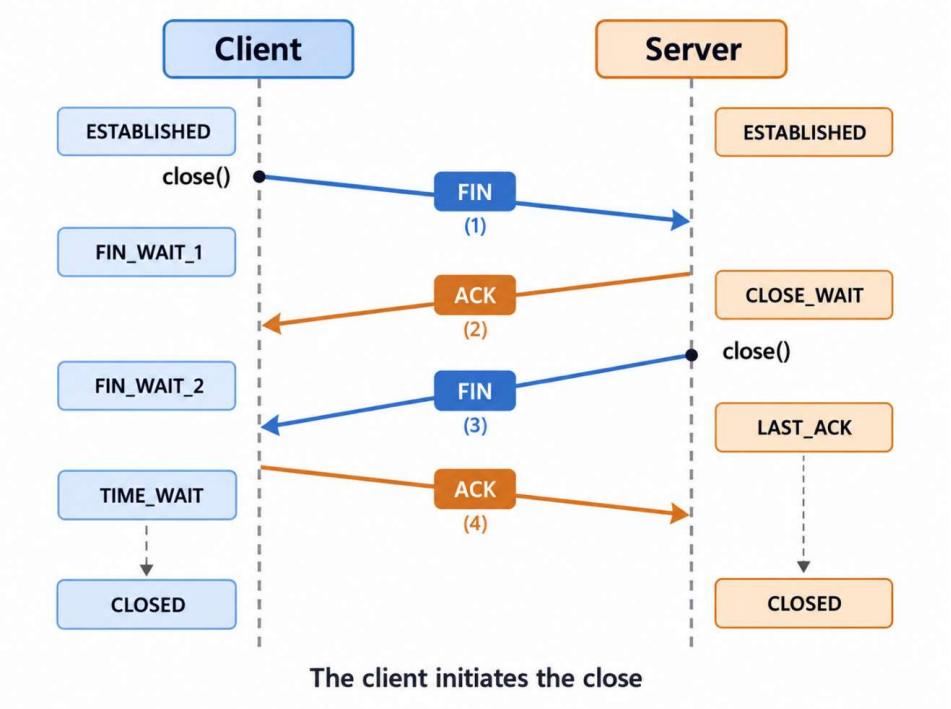
Four-way termination

After sending the final ACK, the endpoint initiating the close enters the TIME_WAIT state.

TIME_WAIT serves two purposes:

- Allows retransmission of the final ACK if the peer retransmits its FIN.
- Allows delayed segments from the old connection to expire before the same connection identifiers are reused.

The connection remains in TIME_WAIT for a limited period.



Sockets: Options SO_REUSEPORT

SO_REUSEPORT is a socket-level option (not part of the POSIX standard).

Purpose:

- Allows multiple sockets to bind to the **same IP address and port**.
- With TCP, multiple sockets can independently call listen() on the same endpoint.
- The kernel distributes incoming connections among these listening sockets, providing built-in load balancing.
- The option must be enabled on each socket before calling bind().

When is it useful?

- Load balancing across multiple processes or threads.
- High-performance, multicore servers.
- Reducing contention on a single listening socket.

Note: On Linux, processes sharing the same address and port must have the same effective user ID.

Sockets: Options SO_REUSEPORT

SO_REUSEPORT is a socket-level option (not part of the POSIX standard).

The kernel distributes incoming connections among these sockets, providing built-in load balancing.

How it works:

- The kernel organizes compatible sockets into a **reuseport group**.
- Incoming TCP connections are distributed among the listening sockets, typically using hash-based selection on Linux.
- Each listening socket has its own accept queue.

Sockets: Send and Receive (TCP)

- Processes of sending or receiving messages in TCP are performed by the read(), write(), send(), receive() functions (simpler than UDP ones):

Stessa interfaccia per scrittura su file:

- Si possono passare le sd a processi figli che possono non distingure tra sd e fd
- Possono essere usati anche con processi che lavorano su file locali.
- Esistono altre funzioni specifiche per leggere e scrivere su socket: 3 modalità di send, 3 modalità di recieve.

Sockets: Send and Receive (TCP)

- Processes of sending or receiving messages in TCP are performed by the read(), write(), send(), receive() functions (simpler than UDP ones):

```
#include <unistd.h>
ssize_t write( int fd,
               const void *buf,
               size_t count);
```

- invia il contenuto del buffer `buf` al socket specificato
- si usa esclusivamente con `SOCK_STREAM`
- restituisce il numero di byte inviati oppure -1 in caso di errore
- è la stessa funzione che consente la scrittura su file

Sockets: Send and Receive (TCP)

- Processes of sending or receiving messages in TCP are performed by the read(), write(), send(), receive() functions (simpler than UDP ones):

```
#include <unistd.h>
ssize_t read( int fd,
              void *buf,
              size_t count);
```

- solo per socket connessi (SOCK_STREAM)
- legge un messaggio di lunghezza massima `len` dal socket
- se non c'è alcun messaggio, il programma rimane sospeso
  - chiamata *bloccante*
- la funzione ritorna il numero di byte letti, `-1` in caso di errore
- è la stessa funzione che consente la lettura da un file

Sockets: Send and Receive (TCP)

- Processes of sending or receiving messages in TCP are performed by the read(), write(), send(), receive() functions (simpler than UDP ones):

Send & Recv:

send è come write (richiede connessione stabilita), ma ha flag per specificare modalità di scrittura,

sendto permette la scrittura su connectionless socket,

sendmsg per specificare buffer multipli

recv come read con flag,

recvfrom per ottenere indirizzo della fonte,

rcvmsg per ricevere da buffer multipli

Sockets: Send and Receive (TCP)

- Processes of sending or receiving messages in TCP are performed by the read(), write(), send(), receive() functions (simpler than UDP ones):

```
#include <sys/types.h>
#include <sys/socket.h>
int send(int s,
         const void *msg, int len,
         unsigned int flags);
int sendto(int s, const void *msg, int len,
           unsigned int flags,
           const struct sockaddr *to, int tolen);
```

man 2 send

- `send()` può essere utilizzata solo se `s` è stato connesso; `sendto()` sempre perché richiede di specificare l'indirizzo di destinazione
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente trasmessi
- `flags` si può lasciare a zero

Sockets: Send and Receive (TCP)

- Processes of sending or receiving messages in TCP are performed by the read(), write(), send(), receive() functions (simpler than UDP ones):

```
#include <sys/types.h>
#include <sys/socket.h>                    man 2 recv
int recv(int s,
         void *buf, int len,
         unsigned int flags);
int recvfrom(int s, void *buf, int len,
             unsigned int flags,
             struct sockaddr *from,
             int *fromlen);
```

- ricevono in `buf` non più di `len` byte. Se `from` non è `NULL`, la struttura `sockaddr` verrà riempita con l'indirizzo del mittente
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente ricevuti
- `flags` si può lasciare a zero

Sockets: Send and Receive (TCP)

- **Send/Receive**
  - “flags" enabling extra functionalities with respect to write/read:

```
ssize_t send(int sock, const void *buf, size_t len, int flags);
ssize_t recv(int sock, void *buf, size_t len, int flags);
```

flag	Effect
`MSG_DONTWAIT`	Non-blocking
`MSG_PEEK`	read without change in receive queue
`MSG_WAITALL`	wait until len is not reached

- MSG_DONTWAIT on recv() — do not wait data (es. polling)
- MSG_DONTWAIT on send() — do not wait if buffer is full
- MSG_PEEK on recv() — to test data before removing
- MSG_WAITALL on recv() — wait until len is reached
- MSG_NOSIGNAL on send() — avoid SIGPIPE

- without flags same as read/write

TCP Socket Programming (Simple Example C/C++)

```
// client-side (includes omitted)

int main() {
   int sockfd, status;
   char buffer[1024];
   const char *hello = "Hello from client";
   struct sockaddr_in servaddr;

   if ( (sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0 ) {
      perror("socket creation failed");
      exit(EXIT_FAILURE);
   }

   memset(&servaddr, 0, sizeof(servaddr));

   servaddr.sin_family = AF_INET;
   servaddr.sin_port = htons(8080);
   servaddr.sin_addr.s_addr = INADDR_ANY;

   int n;

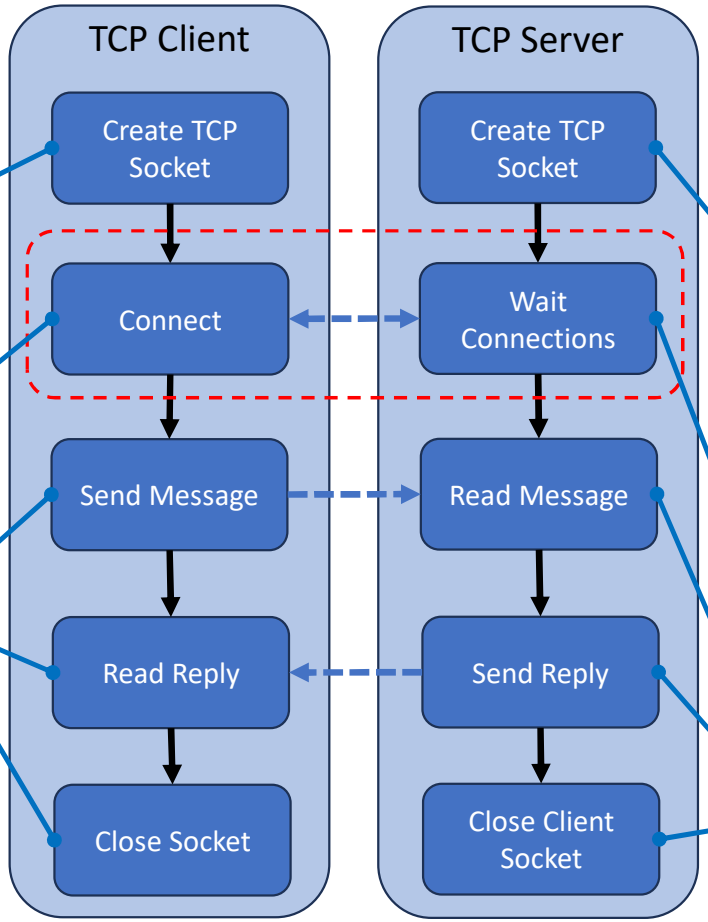
   if ((status = connect(sockfd, (struct sockaddr*)&servaddr,
               sizeof(servaddr))) < 0) {
      printf("\nConnection Failed \n");
      return -1;
   }

   send(sockfd, hello, strlen(hello), 0);
   printf(" Hello message sent \n", buffer);

   n = read(sockfd, buffer, 1024);
   printf("Received \"%s\"\n", buffer);

   close(sockfd);

   return 0;
}
```




```
//server-side (includes omitted)

int main() {
   int sockfd, new_socket;
   char buffer[1024];
   const char *hello = "Hello from server";
   struct sockaddr_in servaddr, cliaddr;

   if ( (sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0 ) {
      perror("socket creation failed");
      exit(EXIT_FAILURE);
   }

   memset(&servaddr, 0, sizeof(servaddr));
   memset(&cliaddr, 0, sizeof(cliaddr));

   servaddr.sin_family    = AF_INET;
   servaddr.sin_addr.s_addr = INADDR_ANY;
   servaddr.sin_port = htons(8080);

   if ( bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0 ) {
      perror("bind failed");
      exit(EXIT_FAILURE);
   }

   if (listen(sockfd, 3) < 0) {
      perror("listen");
      exit(EXIT_FAILURE);
   }
   int addrlen = sizeof(cliaddr);
   if ((new_socket = accept(sockfd, (struct sockaddr*)&cliaddr,
               (socklen_t*)&addrlen)) < 0) {
      perror("accept");
      exit(EXIT_FAILURE);
   }

   int n = read(new_socket, buffer, 1024);
   printf("Received \"%s\"\n", buffer);

   send(new_socket, hello, strlen(hello), 0);
    printf(" Hello message sent \n", buffer);

   close(new_socket);
   close(sockfd);
   return 0;
}
```

Note: to talk to another computer use inet_addr() instead of INADDR_ANY (e.g., inet_addr("192.168.1.10"))

Sockets: Server TCP

- Server-side TCP:

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>     // close

int main(void) {
    int s, c;
    struct sockaddr_in addr = {0};

    addr.sin_family      = AF_INET;
    addr.sin_port        = htons(5200);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    s = socket(AF_INET, SOCK_STREAM, 0);
    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 5);
    c = accept(s, NULL, NULL);
    /* ... comunico con c ... */
    close(c); close(s);
}
```

Sockets: Client TCP

- Client-side TCP:

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>     // close

int main(void) {
    int s;
    struct sockaddr_in addr = {0};

    addr.sin_family = AF_INET;
    addr.sin_port   = htons(5200);

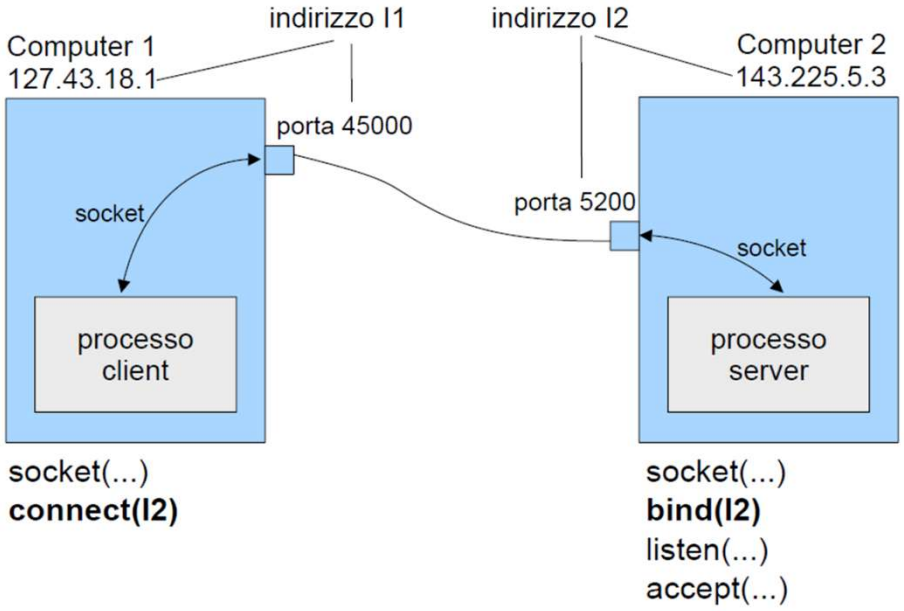
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

    s = socket(AF_INET, SOCK_STREAM, 0);
    connect(s, (struct sockaddr *)&addr, sizeof(addr));

    /* ... comunico con s ... */
    close(s);
}
```

Sockets: TCP connection

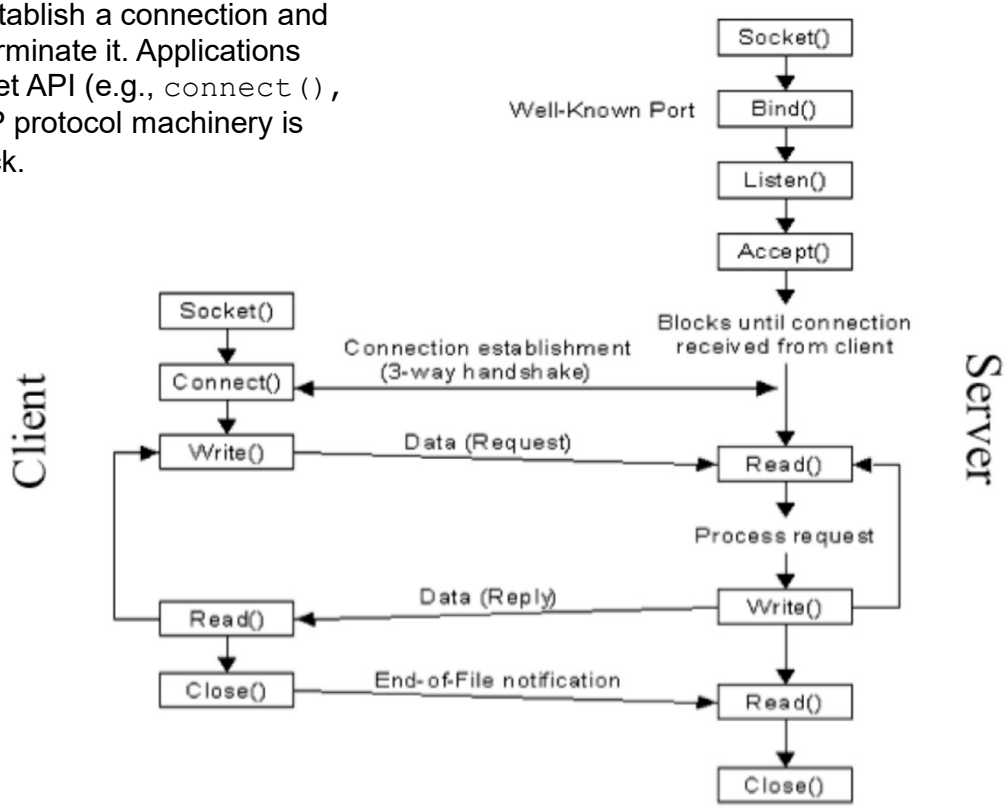
- Socket TCP:



Sockets: TCP connection

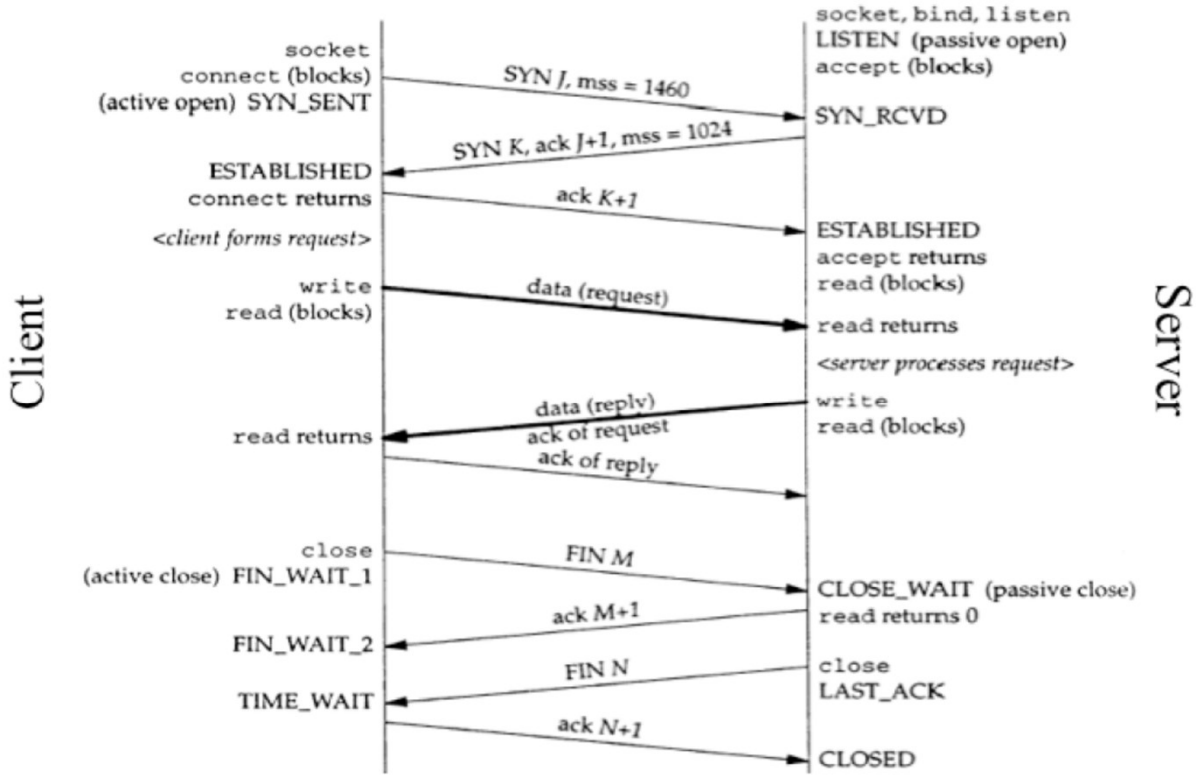
- Socket TCP:

TCP uses a three-way handshake to establish a connection and typically a four-segment exchange to terminate it. Applications interact with TCP only through the socket API (e.g., `connect()`, `send()`, `recv()`, `close()`); the TCP protocol machinery is implemented by the kernel network stack.



Sockets: TCP connection

- Socket TCP:



Sockets: Server TCP

- Server-side TCP: Example – Server increments a number provided by Client

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

static int s = -1;    // listening socket

void handler(int sig) {
    (void)sig;
    if (s >= 0) {
        shutdown(s, SHUT_RDWR);   // Interrupt accept() on Linux
    }
}
```

Sockets: Server TCP

- Server-side TCP: Example – Server increments a number provided by Client

```
int main(void) {

    signal(SIGINT, handler);
    s = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in addr = {0};
    addr.sin_family      = AF_INET;
    addr.sin_port        = htons(5201);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    bind(s, (struct sockaddr*)&addr, sizeof(addr));
    listen(s, 1);

    printf("[server] in ascolto (Ctrl-C per uscire)\n");

    while (1) {
        int c = accept(s, NULL, NULL);
        if (c < 0) {
            // probabile uscita causata da shutdown()
            break;
        }
```

Sockets: Server TCP

- Server-side TCP: Example – Server increments a number provided by Client

```
        uint32_t net_x;
        ssize_t r = recv(c, &net_x, sizeof(net_x), 0);

        if (r == sizeof(net_x)) {
            uint32_t x = ntohl(net_x);
            uint32_t next = x + 1;
            uint32_t net_next = htonl(next);
            send(c, &net_next, sizeof(net_next), 0);
        }

        close(c);
    }

    close(s);
    return 0;
}
```

Sockets: Server TCP

- Client-side TCP: Example – Server increments a number provided by Client

```
int main(void) {

   int s = socket(AF_INET, SOCK_STREAM, 0);

   struct sockaddr_in srv = {0};
   srv.sin_family = AF_INET;
   srv.sin_port   = htons(5201);
   inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);

   connect(s, (struct sockaddr*)&srv, sizeof(srv));

   int x;
   printf("Inserisci un numero: ");

   if (scanf("%d", &x) != 1) {
       printf("Input non valido.\n");
       close(s);
       return 1;
   }
```

Sockets: Server TCP

- Client-side TCP: Example – Server increments a number provided by Client

```
uint32_t net_x = htonl((uint32_t)x);
    send(s, &net_x, sizeof(net_x), 0);

    uint32_t net_res;
    ssize_t r = recv(s, &net_res, sizeof(net_res), 0);

    if (r == sizeof(net_res)) {
        uint32_t res = ntohl(net_res);
        printf("Risposta dal server: %u\n", res);
    } else {
        printf("Errore durante recv\n");
    }

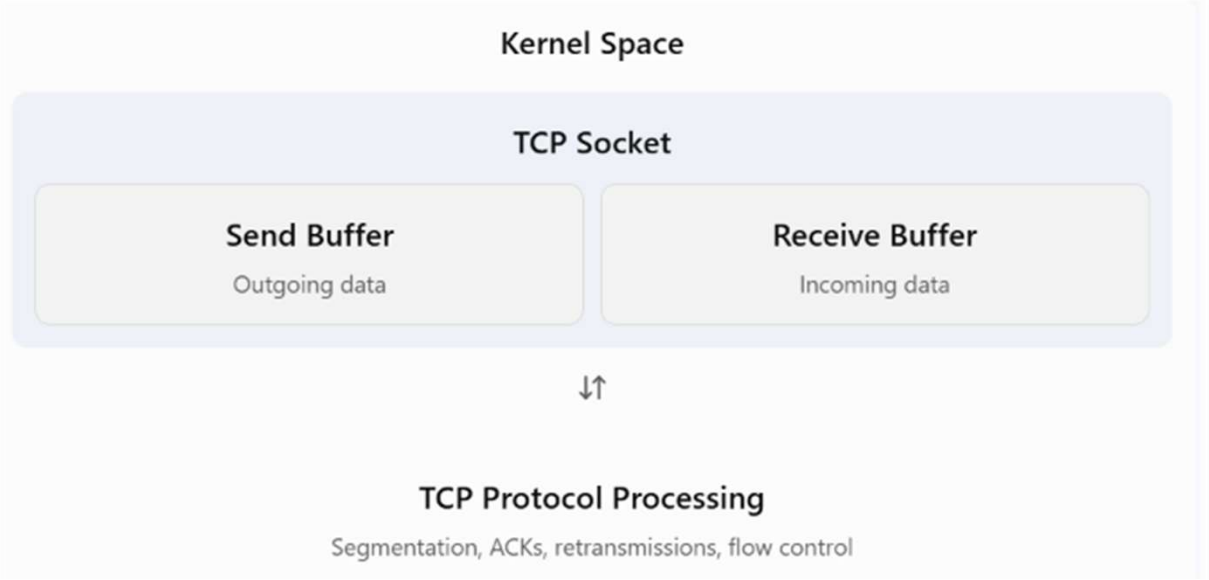
    close(s);
    return 0;
}
```

Sockets: Exercise Client-Server TCP

- Exercise:
  - The client receives 2 numbers from the user
  - The client sends 2 numbers to the server
  - The server sends back their sum to client
  - The client prints the result in stdout
  - The client is ready to read other 2 number from the user

Sockets: Buffer

- TCP sockets have two buffers: a **receive buffer** for incoming data and a **send buffer** for outgoing data. Both temporarily store data in kernel space.



Sockets TCP: Buffer

- The size of this buffer can be configured using the SO_SNDBUF socket option.
- If the available buffer space is smaller than n bytes:
  - **Blocking socket:** write() waits until all requested bytes are accepted into the send buffer. It may return fewer bytes if interrupted or if a send timeout occurs.
  - **Non-blocking socket:** write() never waits for buffer space. It may immediately return a partial byte count or EAGAIN/EWOULDBLOCK.

- In both cases, write() may return fewer bytes than requested. Its return value indicates the number of bytes accepted for transmission, not necessarily delivered to the receiver.
- The default behavior is blocking.

Sockets TCP: Buffer

- **Example**

write(sockfd, buffer, 1000);

Available send buffer space	Blocking write	Non-blocking write
0 bytes	Waits for free space	Returns -1 (EAGAIN/EWOULDBLOCK)
300 bytes	Normally waits to transfer all 1000 bytes	May return 300
1000 bytes	Normally returns 1000	Normally returns 1000

Sockets TCP: Buffer

- When read() is called with n bytes on a TCP socket, the kernel attempts to copy up to n bytes from the socket's receive buffer into the application's buffer.
- If fewer than n bytes are available:
  - **Blocking socket (default):** read() normally returns the available bytes without waiting for the buffer to fill. If no data is available, it blocks until data arrives, the connection is closed, or an error occurs.
  - **Non-blocking socket:** read() returns immediately with the available bytes, or -1 with EAGAIN/EWOULDBLOCK if no data is available.

- The return value indicates the number of bytes actually read. A return value of 0 indicates that the peer has closed its sending side and all pending data has been read.
- The default behavior is blocking

Sockets TCP: Buffer

- **Example**

read(sockfd, buffer, 1000);

Available data	Blocking read	Non-blocking read
0 bytes	Waits for data	Returns -1 (EAGAIN/EWOULDBLOCK)
300 bytes	Normally returns 300	Normally returns 300
1000 bytes	May return 1000	May return 1000

Sockets TCP: Non-Blocking Mode

POSIX method: fcntl()

- Non-blocking mode is enabled by setting the O_NONBLOCK file status flag using fcntl().

```
#include <fcntl.h>
int flags = fcntl(fd, F_GETFL, 0); //retrieves the current file status flags
fcntl(fd, F_SETFL, flags | O_NONBLOCK); //modifies the current file status flags
```

Effects:

- **recv() / read():** return -1 with EAGAIN or EWOULDBLOCK if no data is available.
- **send() / write():** return -1 with EAGAIN or EWOULDBLOCK if data cannot be accepted immediately.
- **connect():** may return -1 with EINPROGRESS while the connection is being established.
- Works with TCP, UDP, pipes, and other supported file descriptors.

Note: MSG_DONTWAIT enables non-blocking behavior for a single send() or recv() call, without changing the socket's mode.

Sockets TCP: Non-Blocking Mode

POSIX method: fcntl()

- Non-blocking mode is enabled by setting the O_NONBLOCK file status flag using fcntl().

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        return 1;
    }

    /* ---- Rendo la socket NON BLOCCANTE ---- */
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags == -1) {
        perror("fcntl(F_GETFL)");
        close(fd);
        return 1;
    }
    if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0)
{
        perror("fcntl(F_SETFL)");
        close(fd);
        return 1;
    }
```

Sockets TCP: SO_SNDBUF / SO_RCVBUF

Each TCP socket has two kernel-managed buffers:

- **Send buffer:** temporarily stores outgoing data received from the application before transmission.
- **Receive buffer:** temporarily stores incoming data before delivery to the application.

Their sizes can be configured using:

- SO_SNDBUF: send buffer size.
- SO_RCVBUF: receive buffer size.

Example:

```
setsockopt(fd, SOL_SOCKET, SO_SNDBUF,
        &size, sizeof(size));

setsockopt(fd, SOL_SOCKET, SO_RCVBUF,
        &size, sizeof(size));
```

Effects:

- **SO_SNDBUF:** a larger buffer can queue more outgoing data, potentially reducing blocking in send().
- **SO_RCVBUF:** a larger buffer can accommodate more incoming data, reducing the risk of TCP flow-control stalls.

Sockets: Safe read

```
#include <sys/types.h>
#include <sys/socket.h>
#include <errno.h>

ssize_t recv_all(int fd, void *buf, size_t n) {
    size_t received = 0;
    char *p = buf; // buffer pointer

    while (received < n) {
        ssize_t r = recv(fd, p + received, n - received, 0);

        if (r < 0) {
            if (errno == EINTR)
                continue;     // try again
            return -1;        // error
        }
        if (r == 0) {
            // close connection ->  return n byte received
            return received;
        }
        received += (size_t)r;
    }
    return (ssize_t)received;   // = n
}
```

Retry on EINTR: the call was interrupted by a signal.
Stop on 0: the peer closed the connection.

Sockets: Safe write

```
#include <sys/types.h>
#include <sys/socket.h>
#include <errno.h>

ssize_t send_all(int fd, const void *buf, size_t n) {
    size_t sent = 0;
    const char *p = buf; //buffer pointer

    while (sent < n) {
        ssize_t w = send(fd, p + sent, n - sent, 0);

        if (w < 0) {
            if (errno == EINTR)
                continue;     // try again
            return -1;        // error
        }

       if (w == 0)
            return (ssize_t)sent;

        sent += (size_t)w;
    }

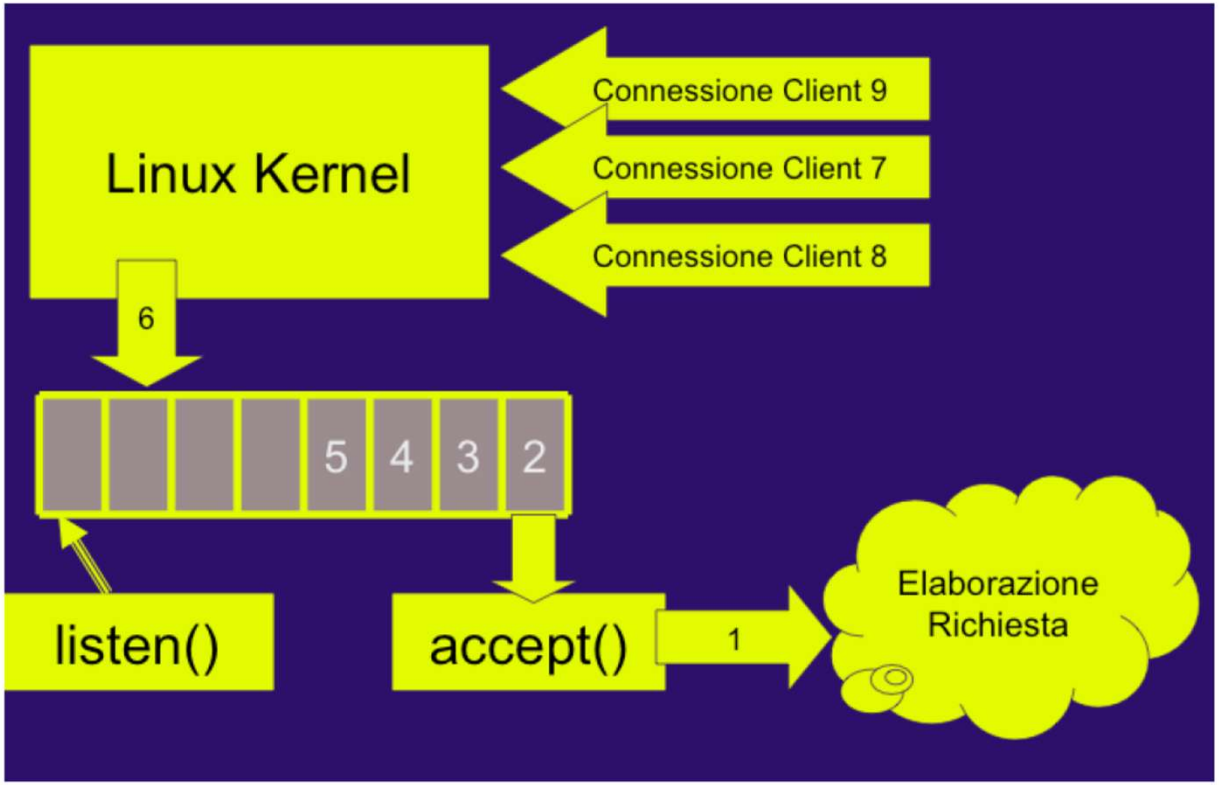
    return (ssize_t)sent;   // = n
}
```

Retry on EINTR: the call was interrupted by a signal.

Sockets: Concurrent Server

- Un generico server attende le richieste di connessione su una determinata porta
- Possono arrivare richieste concorrenti e da molteplici client
- Una soluzione senza programmazione concorrente:
  - client serviti uno alla volta, finché una connessione non è terminata, non vengono serviti altri client interessati al servizio
  - N.B. comunque è il SO che gestisce le richieste concorrenti ed in effetti le serializza

Sockets: Concurrent Server



Sockets: Concurrent Server

- Un server che gestisce sequenzialmente i vari client è insoddisfacente
  - il tempo di attesa di ogni client
    - potrebbe risultare eccessivo
    - dipende dalla durata delle altri connessioni
  - se le connessioni durano molto, il server ben presto diverrebbe indisponibile anche solo ad accodare le nuove richieste di connessioni
- Nei server reali le richieste di vari client devono essere gestite concorrentemente

Sockets: Concurrent Server

- se il server è concorrente, per ogni client viene generato un processo ad hoc per offrire il servizio, in modo che più client possano essere serviti in modalità concorrente. In tal caso l'attesa sulla coda è limitata al tempo necessario alla gestione della richiesta del client da parte del server ed allo start-up del nuovo processo;

Sockets: Concurrent Server

```
socket(...);
bind(...);
listen(...);

while (1) {
        fd2 = accept(fd1, (struct sockaddr *)NULL, NULL);

        if ( (pid = fork()) < 0 ) {
                perror("fork");
                exit(1);
        } else if (pid == 0) {      // processo figlio
                close(fd1); // al figlio non serve fd1
                // gestisce la connessione usando fd2
                ...
                exit(0);    // poi il figlio termina
        }
        // il processo padre chiude fd2 e ripete il ciclo
        close(fd2);
}
```

Sockets: Concurrent Server

```
#define PORT    5201
#define BACKLOG 8

static void handle_client(int fd) {
    char buf[1024];
    for (;;) {
        ssize_t n = recv(fd, buf, sizeof(buf), 0);
        if (n == 0) break;                  // client ha chiuso
        if (n < 0) { if (errno == EINTR) continue; perror("recv"); break;
}
        // echo
        ssize_t off = 0;
        while (off < n) {
            ssize_t w = send(fd, buf + off, (size_t)(n - off), 0);
            if (w < 0) { if (errno == EINTR) continue; perror("send");
close(fd); _exit(1); }
            off += w;
        }
    }
    close(fd);
    _exit(0);  // il figlio termina qui
}
```

Sockets: Concurrent Server

To avoid zombie

```
#define PORT    5201
#define BACKLOG 8

// handler: Reap all terminated child
static void reap(int sig) {// suppress unused parameter warning
    (void)sig;
 // Reap all (-1) terminated child processes, without blocking with no child
    while (waitpid(-1, NULL, WNOHANG) > 0) {}
}

int main(void) {
    // install SIGCHLD handler (SA_RESTART restart accept/recv interrupted)
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = reap;           // function called when SIGCHLD is received
    sa.sa_flags   = SA_RESTART;  // automatically restart interrupted system calls
    sigemptyset(&sa.sa_mask);    // do not block additional signals during handler
    sigaction(SIGCHLD, &sa, NULL); // register the handler for SIGCHLD
```

Sockets: Concurrent Server

Exercise: write the client for this server

Multiple Connections on the Same Port

A TCP server can handle multiple simultaneous connections using a single listening port.

- The server creates a listening socket using socket(), bind(), and listen().
- Each accept() returns a new connected socket, listening socket remains available for new connections.
- All connected sockets share the same local server port.
- Each TCP connection is uniquely identified by a 4-tuple:
  (Source IP, Source Port, Destination IP, Destination Port)

The number of connections is limited by system resources, not by the listening port alone.

Which one?

Limits on Concurrent Connections

A TCP server can handle many simultaneous connections on the same port, but the number is not unlimited.

The main limitations are:

- **File descriptors:** Each connected socket requires a file descriptor. Operating systems impose per-process and system-wide limits.
- **Kernel memory:** Each TCP connection requires control structures and send/receive buffers.
- **CPU resources:** Managing many active connections increases processing overhead.
- **Client source ports:** Connections from the same client IP to the same server IP and port require distinct source ports (16-bit port numbers).
- **Server architecture:** A process-per-client or thread-per-client model consumes more resources than event-driven I/O multiplexing.

Note: Source port exhaustion limits connections from a particular client IP to a specific server endpoint, not the total number of connections a server can accept.

Sockets: Concurrent Server

Server multithreaded

```
socket(...);
bind(...);
listen(...);

while (1) {
  client_len = sizeof(client_addr);
  connect_sd = accept(listen_sd,
                      (struct sockaddr *) &client_addr,
                      &client_len)

   thread_sd = (int *) malloc(sizeof(int));

   *thread_sd =  connect_sd;
   printf("server: new connection from %d \n",connect_sd);
   pthread_create(&tid, NULL, gestisci, (void *) thread_sd);

}
```

Sockets: Concurrent Server

Server multithreaded

```
static void *gestisci(void *arg) {
    int sd = *(int *)arg;
    free(arg);                    // non serve più il contenitore

    char buf[1024];
    for (;;) {
        ssize_t n = recv(sd, buf, sizeof(buf), 0);
        if (n == 0) break;        // client ha chiuso
        if (n < 0) { if (errno == EINTR) continue; perror("recv"); break; }

        // echo (gestione parziale minima)
        ssize_t off = 0;
        while (off < n) {
            ssize_t w = send(sd, buf + off, (size_t)(n - off), 0);
            if (w < 0) { if (errno == EINTR) continue; perror("send"); close(sd); return NULL; }
            off += w;
        }
    }
    close(sd);
    return NULL;
}
```

Sockets: Concurrent Server

Server multithreaded

```
static void *gestisci(void *arg) {
    int sd = *(int *)arg;
    free(arg);                    // non serve più il contenitore

    char buf[1024];
    for (;;) {
        ssize_t n = recv(sd, buf, sizeof(buf), 0);
        if (n == 0) break;        // client ha chiuso
        if (n < 0) { if (errno == EINTR) continue; perror("recv"); break; }

        // echo (gestione parziale minima)
        ssize_t off = 0;
        while (off < n) {
            ssize_t w = send(sd, buf + off, (size_t)(n - off), 0);
            if (w < 0) { if (errno == EINTR) continue; perror("send"); close(sd); return NULL; }
            off += w;
        }
    }
    close(sd);
    return NULL;
}
```

Sockets: Concurrent Server

Server multithreaded

```
for (;;) {
        int connect_sd = accept(listen_sd, NULL, NULL);
        if (connect_sd < 0) { if (errno == EINTR) continue; perror("accept"); continue; }

        // allochiamo un int per passarlo al thread in modo sicuro
        int *thread_sd = malloc(sizeof(*thread_sd));
        if (!thread_sd) { perror("malloc"); close(connect_sd); continue; }
        *thread_sd = connect_sd;

        pthread_t tid;
        int err = pthread_create(&tid, NULL, gestisci, thread_sd);
         if (err) {
              fprintf(stderr, "%s\n", strerror(err));
              close(connect_sd); free(thread_sd);    continue;}
        pthread_detach(tid);    // nessun join: il thread si "auto-raccoglie"
    }
}
```

malloc() provides a separate socket descriptor value for each thread, avoiding race conditions when passing arguments.
pthread_detach() allows thread resources to be automatically released when the thread terminates, without calling pthread_join().

Socket & Select

- **select(): I/O Multiplexing**
  - Waits for I/O events on **multiple sockets simultaneously**.
  - Allows the application to identify which sockets are ready for reading or writing.
  - Avoids blocking on one socket, while other sockets have data available.
  - Eliminates the need for a separate thread or process for each client.
  - Useful for handling multiple clients in a **single-threaded server**.

Socket & Select

- **select(): I/O Multiplexing**
  - Waits for I/O events on **multiple sockets simultaneously**.
  - Allows the application to identify which sockets are ready for reading or writing.
  - Avoids blocking on one socket, while other sockets have data available.
  - Eliminates the need for a separate thread or process for each client.
  - Useful for handling multiple clients in a **single-threaded server**.

Socket & Select

```
#include <sys/select.h>
int select(int nfds, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

- Monitors multiple file descriptors for **I/O readiness**.
- Blocks until one of the following conditions occurs:
  - A descriptor becomes ready for reading or writing.
  - An exceptional condition occurs on a monitored descriptor.
  - The specified timeout expires.
  - A signal interrupts the call.

Return value:

- -1: an error occurred (errno is set).
- 0: the timeout expired.
- Positive value: the total number of ready descriptors reported across the returned sets.

Socket & Select

```
#include <sys/select.h>
int select(int nfds, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

- Monitors multiple file descriptors for **I/O readiness**.
- Blocks until one of the following conditions occurs:
  - A descriptor becomes ready for reading or writing.
  - An exceptional condition occurs on a monitored descriptor.
  - The specified timeout expires.
  - A signal interrupts the call.

Return value:

- -1: an error occurred (errno is set).
- 0: the timeout expired.
- Positive value: the total number of ready descriptors reported across the returned sets.

Socket & Select

File descriptor sets to monitor:

- **readfds:** descriptors to monitor for read readiness.
  - Example: a listening socket is ready when a connection can be accepted using accept().
- **writefds:** descriptors to monitor for write readiness.
- **exceptfds:** descriptors to monitor for exceptional conditions (often set to NULL).

File descriptor sets (fd_set):

- readfds, writefds, and exceptfds are pointers to fd_set variables.
- fd_set represents a set of file descriptors, typically implemented as a bitmask.

Parameter nfds:

- Specifies the range of descriptors examined by select().
- If maxfd is the highest descriptor being monitored: nfds = maxfd + 1
- FD_SETSIZE defines the capacity of an fd_set (typically 1024 on Linux).

Socket & Select

Macros used to manage fd_set:

Function	Description
`FD_SET(int fd, fd_set *set);`	Add `fd` to the `set`.
`FD_CLR(int fd, fd_set *set);`	Remove `fd` from the `set`.
`FD_ISSET(int fd, fd_set *set);`	Return true if `fd` is in the `set`.
`FD_ZERO(fd_set *set);`	Clear all entries from the `set`.

FD_ISSET(): After select() returns, checks whether a descriptor is ready (0 = not ready, non-zero = ready).

Note: select() modifies the sets; they must be rebuilt before each call.

Socket & Select

Select() identifies descriptors ready for I/O operations without blocking.

A socket is ready for reading (readfds) when:

- Data is available in the receive buffer (SO_RCVLOWAT, default: 1 byte).
- The peer has closed its sending side (EOF).
- A socket error is pending.
- A listening socket has connections waiting for accept().

A socket is ready for writing (writefds) when:

- Sufficient send buffer space is available for a write operation.
- A non-blocking TCP connection attempt has completed (successfully or with an error).
- A socket error is pending.

Note: Write readiness does not guarantee that a large send() will complete without blocking.

Socket & Select

Select() identifies descriptors ready for I/O operations without blocking.

A socket is ready for reading (readfds):

```
fd_set readfds;
char buffer[256];

FD_ZERO(&readfds);
FD_SET(sockfd, &readfds);

// Wait until the socket is readable
int ret = select(sockfd + 1, &readfds, NULL, NULL, NULL);

if (ret > 0 && FD_ISSET(sockfd, &readfds)) {
    ssize_t n = recv(sockfd, buffer, sizeof(buffer) - 1, 0);

    if (n > 0) {
        buffer[n] = '\0';
        printf("Received: %s\n", buffer);
    } else if (n == 0) {
        printf("Connection closed.\n");
    }
}
```

Socket & Select

Select() identifies descriptors ready for I/O operations without blocking.

A socket is ready for writing (writefds):

```
fd_set writefds;
char message[] = "Hello";

FD_ZERO(&writefds);
FD_SET(sockfd, &writefds);

// Wait until the socket is writable
int ret = select(sockfd + 1, NULL, &writefds, NULL, NULL);

if (ret > 0 && FD_ISSET(sockfd, &writefds)) {
    ssize_t n = send(sockfd, message,
                     strlen(message), MSG_DONTWAIT);

    if (n > 0)
        printf("%zd bytes accepted for sending.\n", n);
}
```

Socket & Select

select(): Exceptional Conditions

- exceptfds monitors exceptional conditions on sockets.
- For TCP, it is mainly used to detect **Out-of-Band (OOB) data**.
- OOB allows TCP to signal urgent data using the URG flag.
- On Linux, urgent data can be received using recv() with MSG_OOB.

Example:

```
if (FD_ISSET(c, &efds)) {
    char buf[1];
    int r = recv(c, buf, 1, MSG_OOB);
    if (r > 0)
        printf("OOB data: %c\n", buf[0]);
}
```

Socket & Select

The timeout parameter specifies the **maximum time** select() waits for a file descriptor to become ready.

```
struct timeval {
    time_t      tv_sec;   // Seconds
    suseconds_t tv_usec;  // Microseconds
};
```

Timeout behavior:

- **timeout == NULL** select() blocks indefinitely until a descriptor becomes ready or the call is interrupted.
- **tv_sec == 0 && tv_usec == 0** select() returns immediately without blocking (*polling*).
- **Non-zero timeout**
  - select() waits up to the specified time.
  - Returns earlier if one or more descriptors become ready.

Return values:

- **Positive:** number of ready descriptors reported across the sets.
- **0:** timeout expired.
- **-1:** error or interruption (e.g., EINTR).

Socket & Select

```c
#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

#define STDIN 0  // file descriptor for standard input

int main(void)
{
    struct timeval tv;
    fd_set readfds;

    tv.tv_sec = 2;
    tv.tv_usec = 500000;

    FD_ZERO(&readfds);
    FD_SET(STDIN, &readfds);

    // don't care about writefds and exceptfds:
    select(STDIN+1, &readfds, NULL, NULL, &tv);

    if (FD_ISSET(STDIN, &readfds))
        printf("A key was pressed!\n");
    else
        printf("Timed out.\n");

    return 0;
}
```

Socket: multiplexing server

I/O multiplexing allows a server to monitor multiple sockets simultaneously using a single process or thread:

- The server monitors both the **listening socket** and the **connected client sockets**.

General structure of a server using select():

- Initialize fd_set structures for reading and, if needed, writing.
- Add the listening socket and connected client sockets to the sets.
- Call select() to wait for I/O readiness.
- When select() returns:
  - Accept new connections if the listening socket is ready.
  - Read or write data on ready client sockets.
- Repeat the process in a loop.

Key point: A single-threaded server can manage multiple clients without creating a thread or process for each connection.

Socket: multiplexing client

The client monitors two descriptors for reading:

- **Standard input (stdin):** to read messages typed by the user.
- **Connected socket (sockfd):** to receive messages from the server.

Using select(), the client can respond to whichever descriptor becomes ready, without blocking on the other.

```
void str_cli_echo_sel(FILE *fd, int sockfd)
{
  int               maxd, n;
  fd_set            rset;
  char              sendline[MAXLINE], recvline[MAXLINE];

  FD_ZERO(&rset);           /* inizializza a 0 il set dei descrittori in lettura */
  for ( ; ; ) {
    FD_SET(fileno(fd), &rset);  /* inserisce il descrittore del file (stdin) */
    FD_SET(sockfd, &rset);      /* inserisce il descrittore del socket */
    maxd = (fileno(fd) < sockfd) ? (sockfd + 1): (fileno(fd) + 1);
    if (select(maxd, &rset, NULL, NULL, NULL) < 0 ) { /* attende descrittore pronto
      in lettura */
      perror("errore in select");
      exit(1);
    }
```

Socket: multiplexing client

The client monitors two descriptors for reading:

- **Standard input (stdin):** to read messages typed by the user.
- **Connected socket (sockfd):** to receive messages from the server.

Using select(), the client can respond to whichever descriptor becomes ready, without blocking on the other.

```
/* Controlla se il file (stdin) è leggibile */
if (FD_ISSET(fileno(fd), &rset)) {
  if (fgets(sendline, MAXLINE, fd) == NULL)
    return; /* non vi sono dati perché si è concluso l'utilizzo del client */
  if ((writen(sockfd, sendline, strlen(sendline))) < 0) {
    fprintf(stderr, "errore in write");
    exit(1);
  }
}
```

Socket: multiplexing client

The client monitors two descriptors for reading:

- **Standard input (stdin):** to read messages typed by the user.
- **Connected socket (sockfd):** to receive messages from the server.

Using select(), the client can respond to whichever descriptor becomes ready, without blocking on the other.

```
/* Controlla se il socket è leggibile */
if (FD_ISSET(sockfd, &rset)) {
  if ((n = readline(sockfd, recvline, MAXLINE)) < 0) {
    fprintf(stderr, "errore in lettura");
    exit(1);
  }
  if (n == 0) {
    fprintf(stdout, "str_cli_echo_sel: il server ha chiuso la connessione");
    return;
  }
  /* Stampa su stdout */
 recvline[n] = 0;
 if (fputs(recvline, stdout) == EOF) {
    perror("errore in scrittura su stdout");
    exit(1);
  }
 }
}
```

Socket: multiplexing server

The echo server handles multiple client connections using a single thread, storing connected socket descriptors in the client[] array. It maintains two fd_set variables: **allset** e **rset** (ready fd)

```
/* L'array di interi client contiene i descrittori dei socket connessi */
for (i = 0; i < FD_SETSIZE; i++)
  client[i] = -1;

FD_ZERO(&allset); /* Inizializza a zero l'insieme dei descrittori */
FD_SET(listensd, &allset); /* Inserisce il descrittore di ascolto */

for ( ; ; ) {
  rset = allset;  /* Imposta il set di descrittori per la lettura */
  /* ready è il numero di descrittori pronti */
  if ((ready = select(maxd+1, &rset, NULL, NULL, NULL)) < 0) {
    perror("errore in select");
    exit(1);
  }
  /* Se è arrivata una richiesta di connessione, il socket di ascolto
     è leggibile: viene invocata accept() e creato un socket di connessione */
  if (FD_ISSET(listensd, &rset)) {
    len = sizeof(cliaddr);
```

Socket: multiplexing server

The echo server handles multiple client connections using a single thread, storing connected socket descriptors in the client[] array. It maintains two fd_set variables: allset e rset

```
if ((connsd = accept(listensd, (struct sockaddr *)&cliaddr, &len)) < 0) {
   perror("errore in accept");
   exit(1);
 }

 /* Inserisce il descrittore del nuovo socket nel primo posto libero di client */
 for (i=0; i<FD_SETSIZE; i++)
   if (client[i] < 0) {
     client[i] = connsd;
     break;
   }
 /* Se non ci sono posti liberi in client, errore */
 if (i == FD_SETSIZE) {
   fprintf(stderr, "errore in accept");
   exit(1);
 }
```

Socket: multiplexing server

ready = select(...): number of ready descriptors; once managed --ready and break at 0

```
  /* Altrimenti inserisce connsd tra i descrittori da controllare
     ed aggiorna maxd */
  FD_SET(connsd, &allset);
  if (connsd > maxd) maxd = connsd;
  if (i > maxi) maxi = i;
  if (--ready <= 0) /* Cicla finché ci sono ancora descrittori da controllare */
   continue;
}
/* Controlla i socket attivi per controllare se sono leggibili */
for (i = 0; i <= maxi; i++) {
  if ( (socksd = client[i]) < 0 )
    /* Se il descrittore non è stato selezionato, viene saltato */
    continue;
  if (FD_ISSET(socksd, &rset)) {
  /* Se socksd è leggibile, invoca la readline */
    if ((n = readline(socksd, buff, MAXLINE)) == 0) {
      /* Se legge EOF, chiude il descrittore di connessione */
      if (close(socksd) == -1) {
```

Socket: multiplexing server

Removing a client: when recv() returns 0 or error → close(), FD_CLR(), client[i] = -1.

```
            perror("errore in close");
            exit(1);
            }
            /* Rimuove socksd dalla lista dei socket da controllare */
          FD_CLR(socksd, &allset);
          /* Cancella socksd da client */
          client[i] = -1;
        }
        else /* echo */
            if (writen(socksd, buff, n) < 0 ) {
             fprintf(stderr, "errore in write");
             exit(1);
            }
            if (--ready <= 0) break;
       }
      }
     }
}
```

Socket: broadcast & multicast

- **Broadcast**
  - Send data to all hosts on a local network. No group membership required.

- **Multicast** Send data to hosts that have joined a multicast group.

- Both are IP-level mechanisms, commonly used with **UDP sockets** (not standard TCP).

Broadcast: supported by IPv4.
Multicast: supported by IPv4 and IPv6.
IPv6 has no broadcast: multicast is used instead.

Socket: broadcast

- **Broadcast**
  Send data to all hosts on a local network. No group membership required.

- **IPv4 network-layer**
  it allows a packet to be sent to all hosts on a network.

- **Socket programming**,
  broadcast is typically used with UDP because UDP supports connectionless communication.

- TCP does not support broadcast communication.

Socket: broadcast

- **Broadcast**
  Send data to all hosts on a local network. No group membership required.

- **IPv4 broadcast addresses:**
  - **Directed broadcast:** 192.168.1.255 (depends on the subnet mask, discussed later).
  - **Limited broadcast:** 255.255.255.255 (not forwarded by routers).

- **Sender (UDP):**
  - Create a socket: socket(AF_INET, SOCK_DGRAM, 0).
  - Enable broadcast using setsockopt() with SO_BROADCAST.
  - Send the datagram to a broadcast address and destination port using sendto().
- **Receiver (UDP):**
  - Create a UDP socket.
  - bind() to INADDR_ANY and the destination port.
  - Receive datagrams using recvfrom().

Socket: broadcast

- **Sender**

```
#define BROADCAST_ADDR "255.255.255.255" //broadcast limited

int main() {
    int sock; struct sockaddr_in addr; char *msg = "Hello broadcast!";
    // 1. socket UDP
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) { perror("socket"); exit(1);}

    // 2. Enable broadcast
    int yes = 1;
    if (setsockopt(sock, SOL_SOCKET, SO_BROADCAST, &yes, sizeof(yes)) < 0) {
        perror("SO_BROADCAST"); exit(1); }

    // 3. broadcast address
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET; addr.sin_port = htons(PORT);
    addr.sin_addr.s_addr = inet_pton(AF_INET, BROADCAST_ADDR, &addr.sin_addr);

    // 4. Periodic sendto
    while (1) {
        if (sendto(sock, msg, strlen(msg), 0, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
            perror("sendto"); exit(1);}
        printf("Inviato broadcast: %s\n", msg); sleep(1);
    }
  return 0; }
```

Socket: broadcast

- **Receiver**

```
int main() {
    int sock;
    struct sockaddr_in addr;
    char buffer[1024];

    // 1. Socket UDP
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(1);
    }

    // 2. reuse addr and port
    int yes = 1;
    setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    // 3. Bind on the availabe interface
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

Socket: broadcast

- **Receiver**

```
if (bind(sock, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind");
        exit(1);
    }

    printf("listen broadcast UDP on port %d…\n", PORT);

    // 4. Receive loop
    while (1) {
        int n = recv(sock, buffer, sizeof(buffer)-1, 0);
        if (n < 0) {
            perror("recv");
            exit(1);
        }
        buffer[n] = '\0';
        printf("Recived: %s\n", buffer);
    }

    return 0;
}
```

Socket: multicast

multicast allows UDP datagrams to be sent to a group of receivers identified by a multicast IP address (e.g., 239.255.0.1).

To receive multicast datagrams, a socket must join the multicast group using IP_ADD_MEMBERSHIP.

Receiver:

Create a UDP socket.
Bind to INADDR_ANY and the destination port.
Join the multicast group:

Socket: multicast

multicast allows UDP datagrams to be sent to a group of receivers identified by a multicast IP address (e.g., 239.255.0.1).

To receive multicast datagrams, a socket must join the multicast group using IP_ADD_MEMBERSHIP.

```
struct ip_mreq mreq; //multicast group to join and the local network interface to use

// Multicast group address
inet_pton(AF_INET, "239.255.0.1", &mreq.imr_multiaddr);

// Let the OS select the local network interface
mreq.imr_interface.s_addr = htonl(INADDR_ANY);

// Request to join the multicast group (IPPROTO_IP protocollo IPv4)
setsockopt(sock, IPPROTO_IP, IP_ADD_MEMBERSHIP,
        &mreq, sizeof(mreq));
```

Socket: multicast

sender multicast

```
#define GROUP "239.255.0.1"
#define PORT  5000

int main() {
    int sock;
    struct sockaddr_in addr;
    unsigned char ttl = 1;   // TTL = 1 → LAN locale

    // Socket UDP
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(1);
    }

    // Set TTL multicast: sets the Time To Live (TTL) for outgoing multicast packets
    if (setsockopt(sock, IPPROTO_IP, IP_MULTICAST_TTL,
                   &ttl, sizeof(ttl)) < 0) {
        perror("setsockopt - TTL");
        exit(1);
    }
```

Socket: multicast

sender multicast

```
 // Indirizzo multicast di destinazione
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port   = htons(PORT);
    addr.sin_addr.s_addr = inet_addr(GROUP);

    // Loop di invio
    while (1) {
        char msg[128];
        sprintf(msg, "Ciao dalla VM multicast!");

        if (sendto(sock, msg, strlen(msg), 0,
                   (struct sockaddr *)&addr, sizeof(addr)) < 0) {
            perror("sendto");
            exit(1);
        }
        printf("Inviato: %s\n", msg);
        sleep(1);
    }
    return 0;
}
```

Socket: multicast

receiver multicast

```
 #define GROUP "239.255.0.1"
#define PORT  5000

int main() {
    int sock;
    struct sockaddr_in addr;
    struct ip_mreq mreq;
    char msgbuf[256];

    // Socket UDP
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(1);
    }
    // Per permettere a più client di fare bind sullo stesso addrs
    int reuse = 1;

    setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));
    setsockopt(sock, SOL_SOCKET, SO_REUSEPORT, &reuse, sizeof(reuse));
```

Socket: multicast

ll **receiver multicast**

```
    memset(&addr, 0, sizeof(addr)); //  Bind to UDP port 5000 on all local IPv4 addresses
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);

    if (bind(sock, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind"); exit(1);}

    // Join multicast group
    mreq.imr_multiaddr.s_addr = inet_addr(GROUP);
    mreq.imr_interface.s_addr = htonl(INADDR_ANY);
    if (setsockopt(sock, IPPROTO_IP, IP_ADD_MEMBERSHIP,
                   &mreq, sizeof(mreq)) < 0) {
        perror("setsockopt - IP_ADD_MEMBERSHIP"); exit(1); }

    while (1) {// Ricezione
        int n = recv(sock, msgbuf, sizeof(msgbuf)-1, 0);
        if (n < 0) { perror("recv"); exit(1);}
        msgbuf[n] = '\0'; printf("Ricevuto: %s\n", msgbuf);
    }
    return 0;
}
```

Socket: multicast

When an application joins a multicast group, the operating system and the network cooperate to manage multicast communication.

How does it work?

- **Group membership:** the receiver uses IP_ADD_MEMBERSHIP to request membership in a multicast group.
- **Operating system:** the OS manages the membership request and communicates the necessary information to the local network.
- **Network:** multicast-aware routers can use group membership information to forward packets towards networks with interested receivers.
- **Data delivery:** the sender sends UDP datagrams to the multicast IP address, without knowing the individual receivers.

Key idea: socket options allow applications to request network-level behaviors, while the underlying network handles packet delivery.

We will explore how group membership and multicast forwarding work when we study the network layer.

Socket: multicast

Exercise: group chat via multicast

```
int sock;            // socket UDP multicast, bind, and join to the group
struct sockaddr_in mcast_addr; // address group+port
char buf[1024];
[…]
while (1) {
    fd_set rfds;
    FD_ZERO(&rfds);
    FD_SET(STDIN_FILENO, &rfds);
    FD_SET(sock, &rfds);

    int maxfd = (sock > STDIN_FILENO ? sock : STDIN_FILENO) + 1;

    int ret = select(maxfd, &rfds, NULL, NULL, NULL);
    if (ret < 0) { perror("select"); break;}

   /* ... Manage stdin and socket ... */
}
```

Socket: multicast

Exercise: group chat via multicast

```
if (FD_ISSET(STDIN_FILENO, &rfds)) {
    if (!fgets(buf, sizeof(buf), stdin)) { printf("EOF, esco.\n"); break; }

    size_t len = strlen(buf);
    if (len > 0 && buf[len-1] == '\n') {
        buf[len-1] = '\0'; len--;
    }
    if (len == 0) {
        // empty line: nothing to send
    } else {
        ssize_t n = sendto(sock, buf, len, 0,
                           (struct sockaddr *)&mcast_addr, sizeof(mcast_addr));
        if (n < 0) {
            perror("sendto"); break;
        }
    }
}
```

Socket: multicast

Exercise: group chat via multicast

```
if (FD_ISSET(sock, &rfds)) {
    struct sockaddr_in src;
    socklen_t srclen = sizeof(src);

    ssize_t n = recvfrom(sock, buf, sizeof(buf) - 1, 0,
                         (struct sockaddr *)&src, &srclen);
    if (n < 0) {
        perror("recvfrom"); break;
    }

    buf[n] = '\0';

    char ipstr[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &src.sin_addr, ipstr, sizeof(ipstr));

    printf("[%s] %s\n", ipstr, buf);
}
```

Application Layer
DNS Translation

- As just shown, we can create sockets between two hosts, but **we need to know (IP) address and port of the server**.

- On the Internet, hosts are typically identified by **hostnames rather than by IP** addresses.

- Berkeley sockets (C/C++) only works with IP addresses.
  - This is not surprising as **sockets works on transport layer**, while **hostnames works on application layer**.

- If we are to connect with a host by hostname, **we need to use DNS translation** and get the associated IP address to be passed to the socket.

Application Layer
DNS Translation

- C/C++ offers some functions and structures that perform such translation for us, which are used **to contact DNS servers** and to retrieve addresses.

- *gethostbyname()* is a legacy function used to resolve hostnames into IPv4 addresses through the system resolver.

- The above function **does not return just the IP address**. It returns a *hostent* structure that contains some info about the host:
  - Canonical name.
  - Possible aliases.
  - One or more addresses.

  For new applications, getaddrinfo() is recommended, as it supports both IPv4 and IPv6.

Application Layer
DNS Translation

- The hostent structure (legacy) is defined as follows:

```
#include <netdb.h>

struct  hostent {
    char *  h_name;             // original name of the host (canonical)
    char ** h_aliases;          // NULL-terminated array of aliases
    int     h_addrtype;         // family of the address (typically AF_INET)
    int     h_length;           // length of the address (typically 4 bytes)
    char ** h_addr_list;        // NULL-terminated array of addresses
};
```

- A DNS may provide a list of addresses (which may be randomly ordered if the query is sent to round-robin DNS servers).
  - We can get the address of the first element of the array (h_addr_list[0] or h_addr).

- The resolved IP address can be used to initialize a socket address (sockaddr_in).

Application Layer
DNS Translation

- The gethostbyname() function is defined as follows:

```
#include <netdb.h>

struct hostent *host_info = gethostbyname(const char *name);
```

- Where:
  - name: is a string containing the hostname to be translated.
  - host_info: is a pointer to the hostent structure containing the information about host.
    - This is NULL if error occurred.

- Notice that there is also a similar function *gethostbyaddr()* that returns a hostent structure for a specified IP address.

Application Layer

DNS Translation

```
#include <iostream>
#include <cstring>
#include <string>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <unistd.h>
#include <netdb.h>
#include <stdio.h>
```

- Include libraries to allow DNS translation.

```
int main(int argc, char **argv){
   struct hostent *server_info;
   char *server_name;

   if(argc < 2){
      printf("No hostname specified");
      return 0;
   }
   else {
      server_name = argv[1];
   }

   server_info = gethostbyname(server_name);
   if (server_info == NULL){
      printf("could Not resolve hostname %s\n",server_name);
      return 0;
   }
```

- Initialization.
- Get hostname as argument.
- Invoke DNS and check if a response has been received.

Application Layer
DNS Translation

```
    //plot output
    printf("Canonical name:\n)";
    printf("\t %s \n",server_info->h_name);

    printf("Aliases:\n");
    int i = 0;
    while(server_info->h_aliases[i] != NULL){
        print("\t %s\n",server_info->h_aliases[i]);
        i++;
    }

    printf("IPs:\n");
    i = 0;
    while(server_info->h_addr_list[i] != NULL){
        printf("\t %s\n" , inet_ntoa( *( (struct in_addr *) server_info->h_addr_list[i] ) ));
        i++;
    }

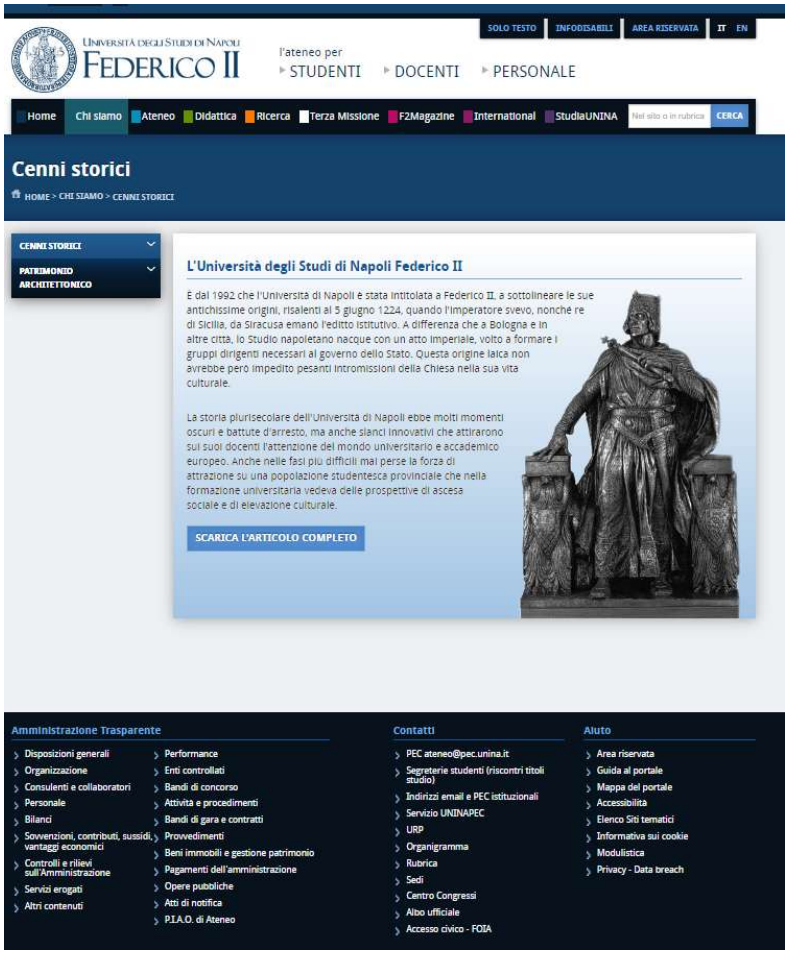
    return 0;
}
```

- Plotting output from hostent structure:
  - Canonical name.
  - List of aliases
  - List of IPs.

Application Layer
HTTP Socket Example

- We will now create a C code implementing from scratch a HTTP request (HEAD) for a web page.

- The web page we will check is the “cenni storici” from the UNINA website:
  - http://www.unina.it/chi-siamo/cenni-storici



Application Layer
HTTP Socket Example

```
#include <iostream>
#include <cstring>
#include <string>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <unistd.h>
#include <netdb.h>
#include <stdio.h>
```

- Include libraries to allow TCP connection to the web server.

```
int main(){
   int socket_desc;
   struct sockaddr_in serv_addr;
   struct hostent *server;
   char buffer[4096];

   socket_desc = socket(AF_INET, SOCK_STREAM, 0);
   if (socket_desc < 0){
      printf("failed to create socket\n");
      return 0;
   }
```

- Initialize variables and create the TCP socket. For a HTTP request 3 elements are specified:
  - Hostname of the web server.
  - Port number.
  - Resource to be retrieved (object's path).

Application Layer
HTTP Socket Example

```
server = gethostbyname("www.unina.it");
if (server == NULL){
   printf("could Not resolve hostname :(" );
   close(socket_desc);
   return 0;
}

memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(80);

 // Copy the binary IPv4 address (stored as a char*)// into the in_addr structure
memcpy(&serv_addr.sin_addr,  server->h_addr_list[0],  server->h_length);

if (connect(socket_desc, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0){
   printf("connection failed :\n");
   close(socket_desc);
   return 0;
}
```

- Connection to the web server. Here we use the following function:

  server = gethostbyname(host.c_str());

  that invokes DNS and to gets the IP of the server from the hostname (inside the *hostent* struct).

- The IP from the returned structure is copied into the *serv_addr* structure for the following connect function.

Application Layer
HTTP Socket Example

```
const char *request = "HEAD /chi-siamo/cenni-storici HTTP/1.1\r\nHost: www.unina.it\r\nConnection: close\r\n\r\n";

if (send(socket_desc, request, strlen(request), 0) < 0){
   printf("failed to send request..." );
   close(socket_desc);
   return 0;
}
printf("message sent:\n" );
printf((«%s\n"  ,request);

while ((n = recv(socket_desc, buffer, sizeof(buffer), 0)) > 0) {write(STDOUT_FILENO, buffer, n);}
```

- Messaging with the web server. The HTTP request is defined into the request string.
- Here we are creating a non-persistent connection since we just want to check one page.
- Close socket and plot the received data.

Application Layer
Socket Example

- getaddrinfo

  getaddrinfo() is a modern function used to resolve hostnames (e.g., www.example.com) into IP addresses (IPv4/IPv6), along with the information needed to create and connect a socket.

  - Resolves a hostname into one or more IP addresses.
  - Supports both IPv4 and IPv6 through a unified interface.
  - Returns a list of struct addrinfo structures containing the information needed by socket() and connect().

  - Usage:

```
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(hints));
hints.ai_socktype = SOCK_STREAM; // TCP

getaddrinfo("www.example.com", "80", &hints, &res);

// res provides addresses ready for socket() e connect()
```

Application Layer
Socket Example

The hints structure specifies the criteria used by getaddrinfo() to select suitable socket addresses.

```
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(hints));

hints.ai_family   = AF_INET;      // IPv4 only
hints.ai_socktype = SOCK_STREAM;  // TCP

err = getaddrinfo("www.example.com", "80", &hints, &res);
```

Main fields of hints:

- ai_family → Address family (AF_INET, AF_INET6, AF_UNSPEC).
- ai_socktype → Socket type (SOCK_STREAM, SOCK_DGRAM).
- ai_protocol → Specific transport protocol (0 = unspecified).
- ai_flags → Additional options controlling address resolution.

Key idea: hints filters the results; res points to the linked list of matching address structures.

Application Layer
Socket Example

- Connecting to a server using getaddrinfo()

```
#define _POSIX_C_SOURCE 200112L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <hostname>\n", argv[0]);
        exit(1);
    }
    struct addrinfo hints, *res, *p;
    char host[INET6_ADDRSTRLEN];
    memset(&hints, 0, sizeof(hints));
    hints.ai_family   = AF_UNSPEC;   // IPv4 + IPv6
    hints.ai_socktype = SOCK_STREAM; // oppure 0 per qualsiasi
```

Application Layer
Socket Example

- Recuperare l'indirizzo del dominio

```
int err = getaddrinfo(argv[1], NULL, &hints, &res);
    if (err != 0) {
        fprintf(stderr, "Errore: %s\n", gai_strerror(err));
        exit(1);
    }
    for (p = res; p != NULL; p = p->ai_next) {
        void *addr;
        const char *ver;

        if (p->ai_family == AF_INET) {
            struct sockaddr_in *ipv4 = (struct sockaddr_in *)p->ai_addr;
            addr = &(ipv4->sin_addr);
            ver = "IPv4";
        } else if (p->ai_family == AF_INET6) {
            struct sockaddr_in6 *ipv6 = (struct sockaddr_in6 *)p->ai_addr;
            addr = &(ipv6->sin6_addr);
            ver = "IPv6";
        } else { continue;}
        inet_ntop(p->ai_family, addr, host, sizeof(host));
        printf("%s: %s\n", ver, host);
    }
    freeaddrinfo(res);
    return 0;
}
```

p->ai_addr can be used directly with connect(), but to access and print the IP address, we must cast it to sockaddr_in (IPv4) or sockaddr_in6 (IPv6).

Application Layer
Socket Example

- Connecting to a server using getaddrinfo()
  Trying multiple resolved addresses

```
struct addrinfo hints, *res, *p;
memset(&hints, 0, sizeof(hints));
hints.ai_socktype = SOCK_STREAM;
hints.ai_family   = AF_UNSPEC; // IPv4 e IPv6

getaddrinfo("www.example.com", "80", &hints, &res);

int sock;
for (p = res; p != NULL; p = p->ai_next) {
    sock = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (sock < 0) continue;

    if (connect(sock, p->ai_addr, p->ai_addrlen) == 0) {
        printf("Connesso!\n"); break; }
    close(sock);
}
if (p == NULL) {
    fprintf(stderr, "Nessun indirizzo si è connesso\n"); exit(1);}

freeaddrinfo(res); // Free the memory dynamically allocated by getaddrinfo()
```

Application Layer

Socket Example

Implement a Simple WHOIS Client in C

Write a C program that:

- Accepts a domain name as a command-line argument (e.g., example.com).
- Resolves the WHOIS server (whois.verisign-grs.com) using getaddrinfo().
- Establishes a TCP connection to port 43.
- Sends the requested domain name followed by \r\n.
- Receives and displays the WHOIS server response.
- Closes the TCP connection.

Application Layer

Socket Example

Implement a Simple WHOIS Client in C

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <netdb.h>
#include <arpa/inet.h>

int main(int argc, char *argv[]) {
    if (argc < 2) return 1;

    struct addrinfo hints, *res;
    memset(&hints, 0, sizeof(hints));
    hints.ai_socktype = SOCK_STREAM;
    getaddrinfo("whois.iana.org", "43", &hints, &res);
```

Application Layer

Socket Example

Implement a Simple WHOIS Client in C

```
int sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    connect(sock, res->ai_addr, res->ai_addrlen);
    freeaddrinfo(res);
    char query[256];
    snprintf(query, sizeof(query), "%s\r\n", argv[1]);
    send(sock, query, strlen(query), 0);

    char buf[1024];
    int n;
    while ((n = recv(sock, buf, sizeof(buf)-1, 0)) > 0) {
        buf[n] = '\0';
        printf("%s", buf);
    }
    close(sock);
    return 0;
}
```